

架构基础概念与方法

Architecture Fundamentals: Concepts and Methods

文档版本: v3.0 **创建日期:** 2026-08-11 **修订日期:** 2026-08-11 **历史:** - 2026-08-11 v1.0→v3.0 从初版迭代形成八章+两附录完整结构: 基本术语与设计链缩写 (§1, 含架构不可视见澄清)、五条基本原理 (§2)、架构五层面目的与意义 (§3, 含系统理论+电商双理论案例)、三层不可逆性判断框架与决策记录六问格式 (§4)、设计链四层决策全景 (§5, 含医学类比+逐层决策表+四层全景对比)、从相关方推导七候选视图 (§6, 含五类生命周期角色+物料供应方条件性纳入)、4A模型×七视图交叉矩阵 (§7, 含三问题/三原因/共存各司其职)、业务架构实践 (§8, BA视图形态与颗粒度+能力描述方法+五层价值+相关方参与+能力成熟度提升循环) ---

阅读指引: 本文档按概念递进组织——从「是什么」到「为什么」到「怎么做」。 - 建立架构认知 → 第 1~3 章 (术语 → 原理 → 目的与意义) - 学习怎么做架构决策 → 第 4 章 (不可逆性判断 + 决策记录格式) + 第 5 章 (设计链各层决策全景) - 搞清楚需要几个架构视图 → 第 6 章 (从相关方推导七候选视图) - 理解企业架构 4A (BDAT) → 第 7 章 (4A 本质、缺陷、与视图的共存关系) - 做业务架构实践 → 第 8 章 (能力地图、颗粒度、五层价值、成熟度提升) - 查术语定义 → 附录 B 术语速查

目录

- [一、基本术语](#)
 - [1.1 系统](#)
 - [1.2 架构](#)
 - [1.3 架构描述、架构视角与架构视图](#)
 - [1.4 架构决策](#)
 - [1.5 相关方与关注点](#)
 - [1.6 术语关系总览](#)
 - [1.7 设计链中的常用缩写](#)
- [二、基本原理](#)
 - [2.1 一个系统，一个架构](#)
 - [2.2 架构决策是不可避免的](#)
 - [2.3 约束即解放](#)
 - [2.4 视图完备性由相关方决定](#)
 - [2.5 架构是对不可逆决策的提前投资](#)
- [三、架构的目的与意义](#)
 - [3.1 认知层面——架构是系统的心智模型](#)
 - [3.2 决策层面——架构划定解空间](#)
 - [3.3 变更层面——架构是系统的免疫系统](#)

- [3.4 组织层面——架构是团队的组织原则](#)
- [3.5 经济层面——架构是一种投资](#)
- [3.6 架构的本质——系统的基本概念](#)
- [3.7 目的与意义总结](#)
- [四、架构决策](#)
- [4.1 不可逆性的三层判断框架](#)
- [4.2 架构决策的记录格式](#)
- [4.3 不可逆性判断的操作化落地](#)
- [五、设计链中各层的架构决策](#)
- [5.1 为什么每层的架构决策不同](#)
- [5.2 需求有架构吗？——「需求方案」的架构](#)
- [5.3 STR 层——定义「需求怎么组织」](#)
- [5.4 BP 层——定义「业务流程及其业务行为单元」](#)
- [5.5 SR 层——定义「系统功能的组织方式」](#)
- [5.6 PA 层——定义「构件的边界和契约」](#)
- [5.7 四层决策全景对比](#)
- [六、架构视图](#)
- [6.1 主流框架对比——为什么没有标准答案](#)
- [6.2 推导方法——从相关方到视图的三步法](#)
- [6.3 七候选视图](#)
- [6.4 相关方 × 视图对应关系](#)
- [6.5 决定视图数量的三个因素](#)
- [6.6 完备性自检——不看数量看覆盖率](#)
- [6.7 追溯——贯穿全部视图的横切面](#)
- [6.8 视图 vs 文档——一个关键区分](#)
- [6.9 视图与设计链的映射——以一个五层链为例](#)
- [七、企业架构4A模型——为什么是这四个？](#)
- [7.1 4A的起源——不是设计出来的，是长出来的](#)
- [7.2 4A就是四个管理地盘](#)
- [7.3 4A × 七视图——两个分类维度的交叉](#)
- [7.4 4A模型的三个问题](#)
- [7.5 4A为什么这么流行](#)
- [7.6 4A与视图的共存——各司其职](#)
- [八、业务架构实践](#)
- [8.1 业务架构视图的形态](#)
- [8.2 业务能力的颗粒度](#)

- [8.3 业务能力的描述](#)
- [8.4 业务架构的五层价值](#)
- [8.5 业务架构与相关方](#)
- [8.6 用业务架构提升能力成熟度](#)
- [附录 A：本文档与通用系统设计规范的关系](#)
- [附录 B：术语速查](#)

一、基本术语

本章定义全文使用的核心术语。定义不引入任何方法论假设——只从概念本身出发，说清楚「是什么」。

1.1 系统 (System)

系统 = 一组相互关联、相互作用的元素，为达成某个共同目的而构成的整体。

(综合自 ISO 9000:2015, §3.5.1 [interrelated or interacting elements]、ISO/IEC/IEEE 15288:2015, §4.1.46 [organized to achieve stated purposes]，及一般系统论 [Bertalanffy, 1968; Ackoff, 1971] 中关于整体性与目的性的论述。四要素框架——元素/关系/边界/目的——为本文档基于上述来源提炼的分析框架。)

一个东西要被称为「系统」，四个要素缺一不可：

要素	含义	缺失的后果
元素	构成系统的组成部分——人、软件、硬件、流程、数据.....	没有元素就没有载体
关系	元素之间的相互作用——数据流、控制流、依赖、协作协议	没有关系，元素只是堆在一起的零件
边界	系统与外界的区分——什么在系统内，什么在外部环境	没有边界就不知道设计范围
目的	系统存在的理由——它要解决什么问题、创造什么价值	没有目的，系统就是无意义的空转

三个重要推论：

- a) **系统是相对的。** 一个系统的元素本身可以是另一个子系统 (subsystem)，系统又存在于更大的超系统 (supersystem) 之中。系统的这种层级嵌套属性，决定了架构也需要分层描述。
- b) **企业本身就是一个系统。** 企业里的人、流程、信息、技术工具、规章制度都是元素，它们通过协作关系 (审批流、汇报链、订单履约链.....) 连接在一起，以「创造和交付价值」为共同目的。
- c) **系统的核心挑战不是「有没有元素」，而是「元素之间的关系是否有序」。** 一堆无序元素不是系统，是混沌。架构要解决的核心问题正是关系的秩序。

1.2 架构 (Architecture)

架构 = 系统在其环境中的基本概念或属性，体现于其元素、关系以及设计和演化的原则之中。
(ISO/IEC/IEEE 42010:2011, 3.2)

拆解这个定义的关键词：

「**基本的**」 (**fundamental**)：架构不关心一切细节，只关心那些如果改变就会让系统变成另一种系统的东西。这划清了架构和详细设计的边界——**架构决策**：修改成本极高、影响范围大、不可逆或几乎不可逆（如选择微服务还是单体、数据所有权按什么域划分）；**详细设计决策**：可在局部范围内修改、不影响系统整体性质（如某个接口用 REST 还是 gRPC、某张表的索引怎么建）。

「**元素和关系**」：架构的两个基本构件。元素 = 系统的组成部分（构件、模块、服务）；关系 = 元素之间的连接（接口、依赖、数据流、调用拓扑）。一个常见的误区是只关注元素而忽略关系——但系统的涌现属性（emergent properties）往往来自**关系**，而非单个元素。可靠性的瓶颈通常不在单个组件，而在组件之间的交互模式（超时重试风暴、级联故障）。

「**设计和演化的原则**」：架构不仅是静态的快照，还包含原则——设计原则约束架构决策如何做出（如「数据所有权按业务域划分」），演化原则约束系统如何随时间变化（哪些可以变、哪些不能变、变更按什么流程走）。这意味着架构不是一张图，而是一个**学习系统**——架构知识随时间和经验持续积累：每次架构层面的变更失误都应沉淀为架构原则的更新。

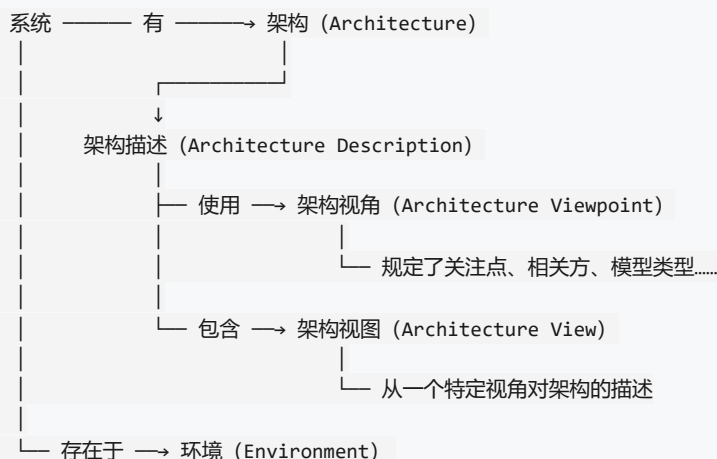
1.3 架构描述、架构视角与架构视图

这是消除概念混淆最关键的一组区分。

一个系统只有一个架构。那些「业务架构」「应用架构」「数据架构」「技术架构」「需求架构」「功能架构」「逻辑架构」「物理架构」……不是多个架构，而是对同一个架构的不同描述——从不同视角看、在不同抽象层次上描述。就像一个人只有一副骨骼，但医生可以从正面拍 X 光、从侧面拍 X 光、从不同层面拍 CT 切片——这些都是同一副骨骼的影像，不是多副骨骼。

混乱的根源在于：日常用语把「架构描述」简称为「架构」，把「架构视图」也简称为「架构」，久而久之，听起来就好像有多个架构了。

ISO/IEC/IEEE 42010:2011 的架构描述框架 (Architecture Description Framework) 给出了精确的术语体系：



概念	是什么	数量
架构 (Architecture)	系统自身的本质属性——它的元素、关系、和原则	一个系统只有一个
架构描述 (Architecture Description)	用来表达架构的文档/制品集合	一个系统可以有一个架构描述，也可以没有（没有文档不意味着没有架构）
架构视角 (Architecture Viewpoint)	一个观察角度——「我从什么立场看这个架构」，规定了关注点、相关方、模型类型	可以定义多个
架构视图 (Architecture View)	从某个视角出发产出的架构描述的一部分	每个视角产生一个视图

为什么日常听到这么多「XX 架构」——两套分类维度的叠加：

市面上各种「XX 架构」实际上来自**两套独立的分类维度**，叠加在一起就产生了混乱。

维度 1：关注域（按「看什么」切分）——TOGAF 等企业架构框架的分类方式：

传统称呼	实际含义	关注的问题
业务架构	从 业务视角 描述架构	业务能力是什么？业务流程是什么？组织怎么分配？
数据架构	从 数据视角 描述架构	数据资产有哪些？数据从哪来、归谁管、怎么用？
应用架构	从 应用视角 描述架构	有哪些应用系统？它们之间怎么交互？
技术架构	从 技术视角 描述架构	用什么技术平台？基础设施怎么部署？

维度 2：抽象层次（按「多详细」切分）——系统工程传统中的分层方式：

传统称呼	实际含义	关注的问题
需求架构 / 概念架构	最抽象的架构描述	系统要做什么？有哪些相关方？
功能架构 / 逻辑架构	中间的架构描述	系统提供什么功能？逻辑数据流是什么？
物理架构 / 部署架构	最具体的架构描述	功能跑在哪些物理节点上？用什么技术实现？

这两个维度存在**重叠和交叉**。任何一个具体的架构视图都同时位于两个维度的某个交叉点上：

关注域 \ 抽象层次	概念层	逻辑层	物理层
业务	业务能力图	业务流程图	业务组织图
数据	概念数据模型	逻辑数据模型	物理数据模型
应用	应用功能列表	应用交互图	应用部署图
技术	技术选型原则	技术平台架构	网络拓扑图

第三类混淆——架构风格/架构模式：

「微服务架构」「事件驱动架构」「分层架构」「六边形架构」.....这些**不是架构，是架构模式/架构风格——即架构决策的套路**。相当于建筑中的框架结构、剪力墙结构、钢结构——这些是结构体系的选择，不是建筑本身。

三种说法的辨析总结：

说法	实际含义	类比
「业务架构 + 应用架构 + 数据架构 + 技术架构」	同一个架构的四个关注域视角（按管理归属切分）	一栋楼的平面图 + 结构图 + 管线图 + 立面图
「需求架构 → 功能架构 → 逻辑架构 → 物理架构」	同一个架构从抽象到具体的四个描述层次	一栋楼的概念草图 → 方案设计 → 扩初设计 → 施工图
「微服务架构 + 事件驱动架构 + 分层架构.....」	架构模式/架构风格——架构决策的套路	框架结构、剪力墙结构、钢结构——结构体系选择

一个自然会问的问题：如果那些都是视图，架构本身长什么样？

答案是：**架构不长什么样**。"长什么样"是视觉范畴的问题，而架构是逻辑范畴的存在——它是一组**基本决策及其衍生的约束关系**，真实存在但不可直接视见。

这听上去有点玄，但其实日常生活里到处都是这种东西。宪法长什么样？你看到的永远是宪法文本，宪法本身是一套治理规则。DNA 长什么样？你看到的是双螺旋结构图，DNA 本身是碱基序列中编码的生命信息。一个公司的组织架构长什么样？你看到的永远是组织架构图，真正的组织架构是汇报关系、决策权限、协作路径——这些东西真实存在，但没有"样子"。

架构也是同一类存在。你能画出来的永远是架构描述——从某个视角、回答某组关注点的投影：

架构（不可视见——组基本决策及其约束）

|

不等于你能画出来的任何一张图

|

└ 通过架构描述（文档/图）来表达

|

└ 系统上下文视图：画出来的是边界和外部交互

|

└ 功能/行为视图：画出来的是功能分解和动态协作

|

└ 构建/开发视图：画出来的是构件清单和接口契约

|

└

只有投影，没有"原图"。

反过来说，如果有人问"给我看一下你们系统的架构"——他能得到的永远是一组视图（边界图、构件图、数据模型.....），而不是一张叫"架构"的图。这不是因为你不肯给，而是因为**不存在这样一张图**。架构是对系统的基本决策的总和——它真实存在于每一个影响半径大、持续时间长、修改成本高的选择之中——但它没有视觉形态。

这个区分不是故弄玄虚，它有实际后果。如果把架构等同于某张图，那么图过时了，人就会觉得"架构过时了"——而实际上过时的只是那张图，架构决策本身（数据按域划分、构件间通过接口通信、变更通过追溯链路评估影响）可以持续有效。图可以烂掉，决策可以活着。

1.4 架构决策（Architecture Decision）

架构决策 = 你在设计系统时做出的某个选择，这个选择选错了的话，后来要花很大代价才能改对。

反过来，如果你做了一个选择，后来发现错了，改一下只要几分钟——这不叫架构决策，这就是普通的实现选择。

三个让人立刻有感觉的例子：

例子 1：书架怎么分类。 1000 本书，按颜色、按作者首字母、还是按主题？选了按颜色——以后每次找「Python 相关的书」都要把所有颜色层翻一遍。选了按主题——「计算机」那一层秒找到。选了之后再想从按颜色改成按主题，500 本书全部重新摆——这就是修改成本。「书架按什么规则分类」就是一个架构决策。

例子 2：厨房的布局。 商用厨房里，灶台和冰箱之间的距离。灶台紧挨冰箱 = 效率高但散热互相影响；灶台和冰箱在对角线 = 各自独立但每次拿食材走 8 步。不管选哪个，一旦灶台接好燃气、冰箱接好水电，再想挪位置就得拆墙重铺管线停工几天——这就是不可逆。

例子 3：软件系统中订单服务的边界。 「库存扣减」放在订单服务里还是库存服务里？放在订单服务 = 前期开发快，但三年后采购入库、盘点调整都要改订单服务，订单服务开始管库存的事，职责膨胀。放在库存服务 = 多一次远程调用，但库存的所有操作（扣减、入库、盘点、调拨）都在一个地方，职责干净。「库存扣减放在哪个服务里」就是一个架构决策。

架构决策有三个标志：

标志	含义	反面（非架构决策）
影响半径大	改了它，不止改一处，要联动改很多东西	改它只影响一个文件、一个函数、一个模块内部
持续时间长	一旦选定，后续无数个其他决策都建立在这个决策之上	今天做了，下周就可以换掉，不影响其他东西
修改成本高	选错了要改对，代价大到你会选择「算了，就忍着吧」	选错了，改对只需要一次 commit

用这三个标志套一下常见决策：

决策	影响半径	持续时间	修改成本	是不是架构决策？
系统拆几个服务，每个服务管什么	大（所有后续开发都在这个框里做）	系统整个生命周期	极高（数据迁移+团队重组+接口重定）	✅ 是
订单服务的数据库选 MySQL 还是 PostgreSQL	中（影响一个服务的 schema 设计+运维）	系统生命周期（迁移成本高）	中-高	✅ 是
订单服务的内部代码用哪个 ORM 框架	小（只影响订单服务内部实现）	中（换框架要重写 DAO 层但接口不变）	中（机械劳动，风险可控）	⚠️ 边界
订单列表的排序逻辑用冒泡还是快排	极小（只影响一个查询的性能）	短（随时可以优化）	极低（换算法一次 commit）	❌ 不是
变量名叫 <code>orderList</code> 还是 <code>orders</code>	极小	中（命名规范一旦确立会持续使用）	低（全局替换即可）	❌ 不是

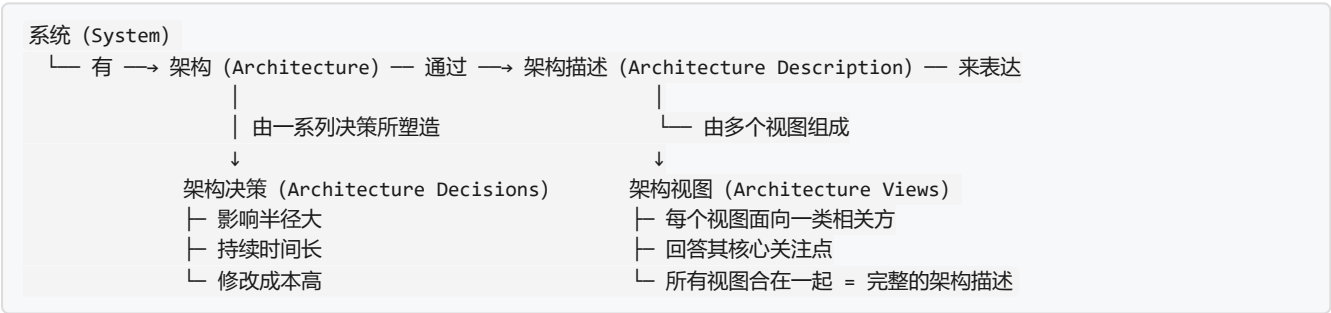
从经济学的角度看：

不可逆性不是技术属性，是经济属性。一个决策是否可逆，取决于修改它需要多少人、多少时间、冒多大风险。「不可逆」不等于「技术上无法修改」——几乎任何软件决策理论上都可以修改。它的真正含义是：修改成本已经高到一个临界点，以至于理性的决策者会选择「忍受现有架构的不完美」而不是「掏出修改成本」。

1.5 相关方与关注点

术语	定义
相关方 (Stakeholder)	与系统有利益关系的个人、团队或组织。相关方是架构视图的出发点和归宿——视图因相关方的需要而存在
关注点 (Concern)	相关方对系统关心的某个方面。一个相关方可以有多个关注点，一个关注点可能被多个相关方共享

1.6 术语关系总览



1.7 设计链中的常用缩写

本文档在讨论设计链时使用以下缩写。这些缩写描述的是设计链中各层的**产物类型**，不依赖任何特定方法论：

缩写	全称	一句话
OR	原始需求 (Original Requirements)	零散的需求表述，未经结构化——「患者说的症状」
STR	相关方需求 (Stakeholder Requirements)	按相关方组织后的结构化需求体系——「医生分诊后的病历」
FR	功能需求 (Functional Requirements)	描述系统"做什么"的需求——行为、功能、业务能力。「FR/NFR 分轨」是设计链中 STR 层和 SR 层的顶层切分维度
NFR	非功能需求 (Non-Functional Requirements)	描述系统"做得怎么样"的需求——性能、安全、可用性、合规等质量属性。NFR 独立于功能需求分轨，保证质量关注点不被功能分解淹没
STR-F	相关方需求—功能需求	STR 的功能需求分支，按用户角色 × 触发组织
STR-NFR	相关方需求—非功能需求	STR 的非功能需求分支，按角色 → 类目 → 分类组织

缩写	全称	一句话
BP	业务流程 (Business Process)	将功能需求转译为业务行为单元（输入→处理→输出，IPO）——「症状拆成可诊断的单元」
SR	系统需求 (System Requirements)	将 BP 的业务行为单元转化为系统功能点和技术约束——「确诊并开检查单」
SR-F	系统需求—功能需求	SR 的功能需求分支
SR-NFR	系统需求—非功能需求	SR 的非功能需求分支，从按角色切换为按分类组织
PA	产品架构 (Product Architecture)	将系统功能归并到可实现的产品构件——「开药方」

这些缩写指代的是**设计链中的某一层产出**，不是某一个具体文档。完整的设计链为：**OR → STR → BP → SR → PA**，每一层都有自己独特的架构决策类型（详见第五章）。

二、基本原理

本章从基本术语出发，建立关于架构的几条基本原理。这些原理回答「为什么架构重要」「架构在做什么」「怎么判断架构做对了」。

2.1 一个系统，一个架构

原理陈述：一个系统只有一个架构。市面上所有的「XX 架构」都是对同一个架构从不同视角、在不同抽象层次上的描述（即架构视图），不是多个独立的架构。详见 §1.3 的完整辨析。

为什么这条原理重要：如果以为系统有多个架构，设计工作就会变成「每个架构画一张图，各画各的」——结果是一个系统有多套互不连通的描述，重要的东西可能在所有「架构」的缝隙中漏掉。正确的认知是：**你只有一套架构，你需要的是从不同角度把它描述清楚。**

2.2 架构决策是不可避免的

原理陈述：架构决策不是「做不做」的问题，而是「什么时候做」的问题。你只有两个选择——在成本低的时候主动做，或在成本高的时候被动做。

用一个思想实验来说明。假设你要盖一栋 30 层的办公楼：

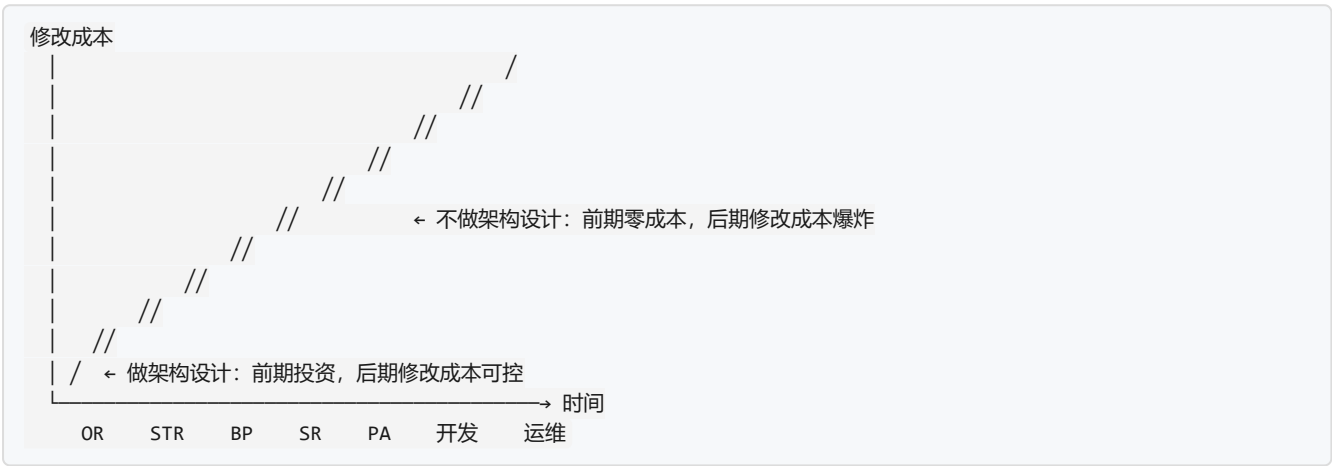
- **做法 A：**找几个施工队直接开工。地基挖到一半发现下面是软土层，得加深——返工。墙砌到 5 层发现电梯井位置和管线打架——拆墙重来。封顶后发现消防通道宽度不达标——改不了。
- **做法 B：**先出结构图、管线图、消防图，各专业对图协调，确认无误后按图施工。

做法 A 并不是「没有架构」——它只是把架构决策**推迟到了施工过程中**。地基要多深、承重墙在哪、管线怎么走、电梯井位置在哪——这些决策一个也跑不掉，终究要做。做法 A 的问题不是没有架构，而是**架构决策发生得太晚**——晚到修改成本已经爆炸。

映射回软件/企业系统：

盖楼	做系统
地基	数据模型——改一张核心表的字段类型，成本可能波及几十个服务
承重结构	服务边界——拆分或合并服务，比改内部逻辑贵一个数量级
管线	接口契约——改了上游接口，所有下游全得适配
消防规范	安全合规——事后改造安全模型，往往意味着架构级推翻重做

修改成本曲线说明了这一点：



为什么这条原理重要：它把架构从「要不要」变成了「怎么做好」——消除了「我们不需要架构」这个伪选项。不画架构图不等于没有架构决策。数据存在哪个库、功能拆成几个服务、接口怎么定义——这些决策在写第一行代码时就在发生，只是做这些决策的人可能没意识到它们的不可逆性。

2.3 约束即解放

原理陈述：架构约束不是限制，而是**回收积极自由**。通过消除低价值的决策空间，架构把注意力集中到真正需要创造力的地方。

区分两种自由：

- **消极自由** (freedom from)：不受约束——「我想怎么做都行」
- **积极自由** (freedom to)：有能力高效完成——「我知道怎么做是对的」

没有架构的时候你拥有消极自由（可以随便加服务、随便定义接口、随便放数据），但你丧失了积极自由——每个决策都需要重新推演上下文，永远在「要不要拆服务」「数据放哪个库」「接口怎么定义」这类问题上消耗精力。

架构通过约束来回收积极自由：

架构决策前：
"这个功能的数据应该放哪个库？" → 需要评估 7 个数据库 × 3 种访问模式 = 21 种可能

架构决策后（"订单域的数据全部归属订单服务自有数据库")：
"这个功能的数据应该放哪个库？" → 订单服务自有库。下一个问题。

为什么这条原理重要：它解释了为什么好的架构不是「管得最多」，而是「管得最准」——只约束那些不改会造成灾难的东西，剩下的留给执行者发挥。架构师的核心能力不是「做出正确的决策」，而是「识别哪些决策如果不正确后果很严重」——对不重要的事情不决策，本身就是一种好的架构决策。

2.4 视图完备性由相关方决定

原理陈述：架构需要哪些视图，不由方法论规定，而由相关方及其关注点决定。视图的数量和内容，取决于哪些人需要理解系统、每个人关心什么。

推导路径：

1. 识别重要相关方 → 2. 明确每个相关方的核心关注点 → 3. 按语言和表达方式聚类关注点 → 4. 每个聚类成为一个视图 → 5. 检查：有7

没有标准数字。3 个也好、7 个也好、12 个也好——只要你知道每一个视图存在的理由（「这个视图在回答谁的什么问题」），且没有重要相关方的核心关注点被遗漏，就是完备的。

为什么这条原理重要：它把「我们需要几个架构视图」这个永远吵不清楚的问题，转化成了「我们的相关方是谁、他们关心什么」这个可以具体讨论的问题。前者是方法论之争，后者是项目分析。

2.5 架构是对不可逆决策的提前投资

原理陈述：架构的经济学本质是——识别哪些决策不可逆（早做，多投入），哪些决策可逆（晚做，保留选择权）。好的架构不是「决定一切」，而是「保留在信息充分时再做决定的自由」。这里的关键经济学概念是**期权的价值**：一个决策如果现在信息不够，推迟它本身就是一种价值——等到信息充分时再做，比现在瞎猜然后返工要便宜得多。

决策类型	特征	操作
高不可逆	影响半径大 + 信息成本高 + 修改风险高	早做，充分分析，多人审查
中不可逆	影响半径中等	做初步分析，预留调整空间
低不可逆	修改成本低，可随时改对	快速决定，错了再改

为什么这条原理重要：它提供了架构投资的决策准则——不是所有架构决策都值得深度分析。把分析资源集中在真正不可逆的决策上，而不是均匀撒在每一个选择上。提前决定服务边界（因为改这个很贵）；推迟具体缓存策略的选择（因为现在信息不够，改这个也不贵）；决定数据所有权归哪个域（因为一旦错了数据迁移成本极高）——三类决策，三种投入。

三、架构的目的与意义

前两章定义了架构是什么、架构决策是什么、以及几条基本原理。本章追问一个更深层的问题：**架构到底有什么用？为什么值得投入？**从认知、决策、变更、组织、经济五个层面展开。

3.1 认知层面——架构是系统的心智模型

一个人同时能有效处理的信息量是有限的（经典是 7 ± 2 个组块）。一个真实的企业系统可能有几百个服务、几千张表、上万个接口——远超单个人的认知能力。

架构做的事情，本质上就是**压缩信息**：

原始复杂度：500 个服务 × 200 个接口 × 50 个数据对象 = 不可管理
架构压缩后：12 个业务域 × 每域 5-8 个核心构件 × 域间 3-5 个接口 = 可管理

这不是丢失信息——丢失意味着你无法还原。架构压缩是**保真压缩**：

- 你不需要同时记住 500 个服务的调用拓扑，但你需要知道「订单域→库存域→物流域」这三个域之间的接口契约
- 当一个需求变更请求出现时，你不需要遍历所有 500 个服务，而是先定位域→定位构件→再深入细节

一个好的架构，让你在处理任何具体问题时只需要关心系统的一小部分，而不用把整个系统装进脑子。

从组织层面看，一个没有架构描述的系统，每个人脑子里对系统的理解都不一样。架构提供了**共享心智模型**（shared mental model）：新成员加入时，架构是最有效的入门材料；跨团队协作时，架构定义了各团队的责任边界；决策讨论时，架构提供了共同的语言。

在一条从需求到实现的设计链中，每一层都是对上一层信息的保真压缩——信息不丢失，但组织得更可管理：

层间过渡	压缩方式
OR → STR	相关方需求将零散的原始需求按角色归并，形成结构化的 What
STR → BP	BP 将需求转译为业务行为单元（输入→处理→输出），压缩掉实现细节
BP → SR	SR 将 BP 的业务行为单元转化为系统功能点，从业务视角切换到系统职责视角
SR → PA	PA 将系统功能点归并到可实现的构件，从逻辑设计压缩为物理设计

3.2 决策层面——架构划定解空间

架构的作用不是给出所有答案，而是**划定解空间**——哪些方案是可以接受的，哪些不是。

无架构：解空间 = 无穷大 → 选择瘫痪
有架构：解空间 = 架构约束下的有限空间 → 可管理的设计决策

举例：如果在 PA 层定了「订单处理」是一个独立服务（架构决策），那么后续所有关于订单的详细设计都在这个框里做——你不需要在每次加订单功能时重新争论要不要拆服务。**架构削减的不是工作量，而是决策量。**

这直接关联到 §2.3（约束即解放）和 §2.5（不可逆决策提前投资）——架构约束消除了低价值的决策空间，让你把注意力集中到真正需要创造力的地方。

3.3 变更层面——架构是系统的免疫系统

系统不会静止。需求变、业务变、技术栈变、人员变。**架构不是为静态世界设计的，而是为持续变化设计的。**

架构在变更管理上的作用从三个时间维度来看：

- a) 事前——预判变更的影响范围。** 你在一条需求上改了两个字：「订单确认后**同步**通知仓库」→「订单确认后**异步**通知仓库」。没有架构：需要翻遍代码找到所有通知仓库的地方，漏一个就是一个线上事故。有架构：追溯矩阵告诉你 OR-042 → STR-F-012 → BP-节点 → SR-F-031 → PA-订单服务 + PA-消息中间件。立刻知道要改哪两个构件，以及变更影响波及到哪些下游节点。
- b) 事中——吸收变更而不崩溃。** 不是所有变更都应该触发级联重设计。架构应该定义变更的刚性边界 vs 柔性区域：

变更类型	影响范围	架构应该提供的支持
添加新的需求条目	新增整条映射链路	架构应支持插拔式新增，不破坏现有结构
修改现有需求指标值	仅影响层级链中承接该指标的节点	指标分解的可追踪链路
新增一个业务域	可能影响顶层分类和多个下游节点	域间松耦合，新域不破坏旧域
更换底层技术组件	PA 构件内部实现变更，不影响上层	PA 构件的封装性——内部可替换而不改变接口契约

c) 事后——从变更中学习。 每次架构层面的变更失误都应该沉淀为架构知识的更新——「为什么这个接口契约的设计没有预见到这种使用场景？」→ 下次设计类似接口时，这个教训成为架构原则的一部分。这就是架构定义中「演化的原则」的真正含义——架构不是一张图，而是一个**学习系统**。

3.4 组织层面——架构是团队的组织原则

康威定律 (Conway's Law)：

任何组织设计出来的系统，其结构都是该组织沟通结构的副本。

反过来也成立——**逆康威定律**：

如果你想让系统有某种结构，就先让团队有对应的组织结构。

架构在这里扮演的角色是**协调层**。具体来说，产品架构 (PA) 层的构件边界就是天然的团队边界：

- 如果 PA 定义「订单服务」和「库存服务」是两个独立的构件，它们可以由两个团队独立开发、独立测试、独立部署
- 如果订单和库存被定义为一个构件的两个子模块，那它们大概率在同一个团队里

这就是为什么 PA 构件应满足**三独立**——可独立分配、独立替换、独立验证。这三个「独立」不只是技术属性，也是**组织属性**：可独立分配 = 可以分配给不同团队；可独立替换 = A 团队重构订单服务不影响 B 团队。

3.5 经济层面——架构是一种投资

架构决策的修改成本不是线性的。这里有两个关键经济学概念：

概念 1：不可逆决策的提前锁定。 有些决策一旦做了，修改成本随时间指数增长——数据所有权归属、服务边界、对外 API 契约。这些需要在信息最充分的时候做，并投入足够的分析。

概念 2：期权的价值。 有些决策可以推迟——等到信息更充分时再做，比现在瞎猜然后返工要便宜得多。好的架构师知道哪些决策需要现在锁死、哪些可以留给未来。

提前决定：服务边界（因为改这个很贵）
推迟决定：具体缓存策略（因为现在信息不够，改这个也不贵）
必须早做：数据所有权归哪个域（因为一旦错了，数据迁移成本极高）

3.6 架构的本质——系统的基本概念

回到 ISO 42010 的定义中一个容易被忽略的关键词——「基本概念或属性」。它不是在说架构的内容（元素+关系+原则），而是在说架构的**本体**：

架构不是你画的图、写的文档。架构是系统的「基本概念」——那些决定系统是什么而非其他什么东西的本质属性。

图可以过时，文档可以过时，但如果架构决策本身是正确的（数据所有权按业务域划分、构件间通过明确接口通信、变更通过追溯链路评估影响），那么这些「基本概念」会持续发挥作用，不管当初画的那张图还有没有人维护。

软件思想家 Ralph Johnson（《设计模式》四位作者之一）有一个精辟的表述：

Architecture is about the important stuff. Whatever that is.

这有一个深刻的隐含前提：**「什么是重要的」本身是需要判断的，而判断依据就是你对这个系统的理论。**

所谓「系统的理论」，是你对这个系统的核心运作逻辑的基本判断——它回答三个问题：这个系统靠什么成功？（定位核心价值驱动力）它会面临的**最大压力**在哪里？（识别关键约束点）什么东西做错了最致命？（区分轻重缓急）

举一个具体例子。同样是做一个电商平台：

	架构师 A 的理论	架构师 B 的理论
核心判断	这个平台靠订单吞吐量取胜——双十一峰值是日常的 50 倍，撑不住就没生意	这个平台靠商品发现取胜——用户不是因为没地方下单而流失，而是因为找不到想要的
"什么东西重要"完全不同	订单管道的水平扩展、数据库分片策略、消息队列的削峰能力	搜索索引的实时性、推荐管线的延迟、A/B 实验框架的灵活性
"什么东西可以推迟"完全不同	复杂的推荐算法可以先做一个简单的热榜凑合用	极致的吞吐量可以先靠加机器撑过去，优化的事以后再说

两个架构师面对的是同一个电商系统，但因为对"这个系统靠什么成功"的基本判断不同，他们画的架构图、做的架构决策、优先投入的资源方向完全不同。**这就是"理论"在起作用。**

一个没有理论支撑的架构就像没有物理学原理的工程——你可能碰巧做对，但不知道为什么对，下次换一个上下文就错。而一个好的架构师，不是那个能画出最漂亮架构图的人，而是那个**对自己系统的理论最清醒的人**——他知道自己的判断依据是什么，因此也知道在什么条件下这个判断会失效。

3.7 目的与意义总结

维度	架构的作用
认知	压缩信息，提供共享心智模型——好的架构让你每次只需关心系统的一小部分
决策	约束解空间，消除低价值选择——架构削减的不是工作量，而是决策量
变更	预判影响、吸收变更、从变更中学习——架构是系统在时间维度上的免疫系统
组织	定义团队边界和协作接口——架构是团队的组织原则（康威定律）
经济	用前期分析成本换取后期返工减少，保留可推迟决策的期权价值

一句话收束：

架构是对不可逆决策的提前投资，目的是在系统生命周期中最大化每个决策的认知质量，同时最小化每个决策的修改成本。

四、架构决策

第一章定义了架构决策是什么，第二章和第三章阐述了它为什么重要。本章展开讨论两个操作性问题：**怎样判断一个具体决策的不可逆性？** 以及 **怎样记录架构决策？**

4.1 不可逆性的三层判断框架

「不可逆」不等于「技术上无法修改」。几乎任何软件决策理论上都可以修改。「不可逆」的真正含义是：**修改成本已经高到一个临界点，以至于理性的决策者会选择「忍受现有架构的不完美」而不是「掏出修改成本」。**

判断一个架构决策的不可逆性，从三个维度分别评估：

第一层：修改的影响半径

改这个决策，需要联动修改多少东西？影响半径决定了修改成本的下限。

维度	低耦合（易修改）	高耦合（难修改）
结构耦合度	数据归属明确，改一个域不影响其他域；内部接口，单一消费者	数据跨多个服务共享，改所有权涉及数据迁移；公开 API，几十个外部消费者
时间耦合度	可渐进式迁移——v1 接口保留 3 个月，下游逐步迁移到 v2	必须一次性同步上线——如数据库 schema 变更，所有读写方必须同步升级

维度	低耦合（易修改）	高耦合（难修改）
组织耦合度	修改只需一个团队内部协调	修改需要跨团队对齐、同步排期。技术上的耦合可以通过工具解决，组织上的耦合需要开会

第二层：修改的信息成本

有些决策修改的成本不在「动手改」，而在「重新搞清楚应该改成什么样」。

决策的知识深度：当初这个决策是靠一条长推演链做出来的——跨多层、多域分析权衡后才得出的结论。修改它意味着需要**重走一遍推演链**，重新确认所有下游依赖仍然成立。追溯链路越长，修改的信息成本越高。

决策的隐含上下文：当初做决策的背景信息没有被记录下来，只在当事人的脑子里。当这些人离开、忘记或转岗后：

修改一个决策的真正成本 = 重新发现当初为什么做这个决策的成本 + 实际修改的成本

决策记录（\$4.2）的价值正在于此——降低未来的信息恢复成本。

第三层：修改的风险

有些决策不是不能改，而是一旦改错了后果严重到无法接受。

爆炸半径：被修改的东西是否处于业务关键路径、数据关键路径、安全关键路径的交汇点？

修改内部管理后台的 UI 颜色 → 错了 → 难看，不急
修改核心支付流程的状态机 → 错了 → 资金损失，紧急
修改用户认证的鉴权模型 → 错了 → 安全漏洞，灾难

数据完整性风险：涉及历史数据迁移的修改自动升级一个风险等级：

修改新写入数据的 schema → 低风险（新的按新结构写）
修改已有历史数据的 schema → 中风险（迁移脚本可能出错、可能丢数据）
修改多服务共享的核心数据所有权 → 极高风险（数据迁移 + 多服务同步上线 + 回滚策略复杂）

这就是为什么**数据所有权的划分**几乎是架构中最不可逆的决策——一旦数据被多个服务依赖，修改所有权就像在飞行中给飞机换引擎。

综合判断矩阵

决策示例	影响半径	信息成本	修改风险	综合判定
服务边界划分	大（影响团队组织+接口契约）	高（涉及业务域理解）	高（数据迁移）	高不可逆
对外 API 接口契约	大（外部消费者不可控）	中	高（破坏外部集成）	高不可逆
数据所有权归属	大	高	极高（数据迁移）	极高不可逆
架构模式（微服务 vs 单体）	极大	高	极大	几乎不可逆

决策示例	影响半径	信息成本	修改风险	综合判定
数据库选型	大	中	中（有迁移工具但成本高）	中-高不可逆
编程语言选择	大（全代码库）	低（社区标准明确）	中（重写成本高但可渐进）	中不可逆
内部工具库选型	中	低	低（可逐步替换）	低不可逆
日志格式	小	低	低	高度可逆

4.2 架构决策的记录格式

一个架构决策的完整记录，回答六个问题。下面用一个真实例子来演示——某个 EOS 流水线项目中关于系统设计子线按四种场景模式组织的决策：

决策名称：系统设计子线按四种场景模式（瀑布/敏捷/逆向/DevOps）组织，而非统一流程

背景：EOS 流水线的系统设计子线需要覆盖不同的设计场景——有的是全新系统（需求明确），有的是迭代开发（需求渐进），有的是补文档（已有系统），有的是小变更（快速交付）。一种流程无法同时满足四种场景的效率要求。

选项：

A. 单一流程，通过参数/配置来适配不同场景
好处：简单，一套文档规范
代价：对 DevOps 场景太重（几小时的活走 30 天流程不合理），对瀑布场景又太弱（缺少足够的分析深度）

B. 四种独立模式，各自有独立的任务定义和操作流程 ← 选了它
好处：每种场景匹配最合适的分析深度和周期
代价：需要维护四套任务定义，四种模式之间的切换需要显式决策

选择：B。因为场景之间的效率差异太大（几小时 vs 30天），一套流程无法兼顾——用错了流程的代价 > 维护四套的代价。

影响：系统设计子线有了四个独立子目录（瀑布/逆向/敏捷/DevOps），每个 Skill 需要定义四种模式的变体或注明适用模式。后续新增设计场景时，需要判断是归入现有模式还是新增模式。

不可逆性：高。因为所有下游 Skill 的任务定义都建立在四模式分类之上，如果要改成三模式或五模式，需要重写所有 Skill 的操作流程。

这个六问格式——决策名称、背景、选项、选择、影响、不可逆性——适用于任何架构决策，无论它发生在需求组织层（STR）、业务流程层（BP）、系统功能层（SR）还是产品构件层（PA）。

4.3 不可逆性判断的操作化落地

不可逆性判断不只是理论——它可以嵌入到设计过程中，转化为具体的操作机制。

4.3.1 不可逆性标注

在架构末级节点上标注其不可逆性，使后续维护者能一眼识别修改风险：

PA-订单服务：

不可逆性：高

理由：数据所有权归订单域，涉及订单→库存、订单→支付的接口契约
修改代价：影响 3 个域 × 5 个服务 × 2 个外部系统集成

标注的价值不在当前设计者——当前设计者知道为什么这么选。价值在**一年后换了一个人接手**：他看到「不可逆性：高」就知道不要轻易动这个决策，看到「理由」就知道动了会波及什么。

4.3.2 按不可逆性分级的审查深度

不是所有架构决策都值得同样深度的审查。按不可逆性调整审查投入：

不可逆性	最低审查深度	审查参与者
极高	语义+指标完整审查	多人交叉审查
高	语义+指标完整审查	至少一人独立审查
中	语义符合性	自查 + 同行确认
低	快速自检	自查即可

这个分级审查机制确保**把有限的审查资源集中在真正不可逆的决策上**——而不是均匀撒在所有决策上。

4.3.3 「推迟决策」作为一种显式架构操作

不是所有决策都需要立即做出。当一个决策的不可逆性高但当前信息不充分时，**「明确决定推迟」本身就是一个有效的架构操作**。

这在快速迭代场景中尤其重要。例如 DevOps 模式通常倾向于快速决策，但并非所有变更都是低不可逆的：

DevOps 模式标准流：
收到小变更 → 快速分析 → 实施

嵌入不可逆性预判后：
收到小变更 → 不可逆性预判（30 秒） →
 低不可逆 → 快速实施
 高不可逆 → 标记为「需完整设计推演」→ 升级到更重的分析路径

架构师的核心能力不是「做出正确的决策」，而是「识别哪些决策如果不正确后果很严重」——对不重要的事情不决策，对信息不足的高风险决策主动推迟，本身就是好的架构决策。

五、设计链中各层的架构决策

第四章讨论了架构决策的通用框架——如何判断不可逆性、如何记录决策、如何按不可逆性分级审查。本章将这套框架应用到一条具体的设计链上，逐层分析：在从需求到实现的设计链中，每一层做的架构决策有什么不同？为什么不同？

以下以一条从原始需求（OR）→ 相关方需求（STR）→ 业务流程（BP）→ 系统需求（SR）→ 产品架构（PA）的五层链为例，逐层分析。注意：OR 是纯需求，无架构决策。架构决策从 STR 开始——因为 STR 是第一个「方案」，而方案才有架构。

5.1 为什么每层的架构决策不同

设计的本质是对需求的持续分解、分配和符合性分析，直至产出可实现的方案。在这个过程中，每一层面对的需求抽象层次不同、向下传递的目标对象不同，因此每层做的架构决策天然不同。

可以用一个医学类比来理解：

层	类比	在做什么决策
OR	患者说「我头疼、吃不下饭、睡不好」	不决策，只是表述
STR	医生分诊：头痛→神经内科，吃不下饭→消化内科，睡不好→两个科室都会诊	把散乱的自述归类到科室
BP	神经内科医生：偏头痛还是紧张性头痛？什么时候发作？持续多久？	把笼统主诉拆成可诊断的症状单元
SR	确诊并开检查单：偏头痛，做脑电图+MRI	把症状转化为可检验的医学判断
PA	开药方：普萘洛尔 40mg bid + 布洛芬按需	把判断转化为可执行的处方

每一层都在**承上启下**——把上一层的输出转译为下一层能理解和操作的语言。转译过程中必然要做决策。

但有一个贯穿各层的共性：**每一层的架构决策，本质上都在回答「这部分需求，由谁来承接、以什么形式承接、和其他承接者是什么关系？」**只不过每一层的「承接者」不同——STR 层的承接者是需求分类节点，BP 层的承接者是业务行为单元（IPO），SR 层的承接者是系统功能点，PA 层的承接者是产品构件。

5.2 需求有架构吗？——「需求方案」的架构

这是一个经常引起困惑的问题：「需求架构」这个词听起来像个矛盾——需求不是回答 What 吗？架构不是回答 How 吗？那需求的架构是什么东西？

5.2.1 答案：需求本身没有架构，但需求的方案有架构

OR (原始需求) 是设计链的起点——它是一堆零散的、可能互相重叠甚至矛盾的需求表述。OR 没有架构定义子文档，没有详细定义子文档，只有条目清单。**纯需求没有架构，也不需要架构。**

但当你面对这堆零散需求，要把它们组织成一套结构化的、可验证的、可向上传递的需求体系时（STR），这个组织工作本身就是一次设计——它的输出是**需求的方案**，而方案就需要架构。

不存在「需求的架构」。存在的是「需求方案的架构」——即如何将零散的原始需求组织成结构化需求体系的架构。





OR 没有架构定义子文档，STR 有——这不是偶然的。**OR 是设计链的起点，它必须先存在，才有后面的事；STR 是第一个「方案」——它是「如何组织需求」这个设计问题的答案，而答案需要架构。**

5.2.2 为什么这个区分重要

这澄清了一个深层的概念混淆。日常说「需求架构」时，实际指的是 STR 层的架构——即需求方案（STR）的架构，而非需求本身的架构。STR 层的架构决策（按什么维度分类、按什么颗粒度组织、冲突怎么处理）**不决定「系统怎么做」，但决定了「系统设计从哪里出发、按什么逻辑组织、怎么验证完备性」**。这些决策的影响贯穿整个设计链，不可逆性极高。

这一点和通用系统设计规范 §1.1 中「需求/方案相对」的定义形成呼应：同一份文档在不同层位上兼具不同身份——STR 向上是 OR 的方案，向下是 BP/SR 的需求。这种双重身份正是「需求方案也需要架构」的机制基础。**STR 之所以既有架构定义子文档又有详细定义子文档，正是因为它在设计链中扮演着双重角色——作为方案需要架构来组织，作为需求需要详细定义来传递。**

5.2.3 STR 层的架构决策——一个更深的认识

STR 层的架构决策虽然不涉及任何系统实现细节，但它定义了**系统的概念边界**——系统要解决谁的问题（用户角色）、解决什么问题（需求内容）、做到什么程度算成功（指标）。它不是系统内部的架构，但它是系统的「外部架构」——系统与它的环境（用户、相关方、约束条件）之间的关系结构。

这回到了 §1.1 中「系统」定义四个要素之一——**边界**。STR 层做的架构决策，本质上就是在定义系统的边界：什么东西在系统里（需要被后续层设计）、什么东西在系统外（是需求来源、是约束条件、是不需要系统管的）、系统和外界的交互点在哪。从这个意义上说，**STR 层是整个五层链中唯一同时对「系统内」和「系统外」的层——它既承接了外部相关方的需求输入，又为后续层的内部设计划定了起跑线。**

5.3 STR 层——定义「需求怎么组织」

这一层在解决什么设计问题：一堆零散的、可能重叠或矛盾的原始需求表述（OR），需要被组织成一套结构化的、可验证的、可向上传递的需求体系。

用一个具体例子来理解——假设你做了需求调研，带回来 100 张便签纸：

"订单提交后要能实时看到状态"、"系统不能太慢"、"财务那边要能导出报表"、"审批流程要支持多级"、"数据要安全"、"操作界面要简洁"、"要支持手机端操作"、"库存不足时要能自动提醒".....
(还有 92 条)

你现在要做的就是把它们变成一套可以往下传递的需求文档。你立刻面临一系列决策：

决策 A——按什么维度分类？ 按「谁提的」（采购部/销售部/财务部）？还是按「功能 vs 非功能」？还是按「业务流程阶段」？如果你选了按提需求的人分类，同一条端到端流程会被拆到三个部门的需求堆里——采购员提了「下单要快」、审批者提了「审批要有超时提醒」、仓库提了「发货要能扫码」——这三条其实串在同一条流程上，但按部门分类让你看不出来。

决策 B——每条需求写到什么粒度？ 「系统不能太慢」是一条还是多条？拆成「订单列表页加载 < 2s」「订单提交接口 < 500ms」「审批流转 < 1s」——太粗了下游不知道要优化哪里，太细了需求决策者验证不了（「我怎么知道审批流转是不是 1 秒？我自己用起来不慢就行了」）。

决策 C——冲突的需求怎么组织呈现？ 采购部说「审批要严格，至少三级」，销售部说「审批要快，最好一级过」。两条都保留标注冲突等待裁决？还是预先合并为「审批支持灵活配置 1-3 级」？前者保持了需求原样但给下游增加了处理负担，后者简化了下游但提前做了设计决策（这可能不在需求阶段的权限范围内）。

这三个决策——分类维度、粒度约定、冲突处理规则——**它们都是架构决策，虽然不决定系统怎么做，但决定了整条设计链从哪里出发。**

STR 层的形式化架构决策：

决策	内容	备选方案	为什么架构决策
顶层按什么切分	FR/NFR 分轨	FR 和 NFR 混在一棵树上	NFR 独立分轨可以独立验证完备性，混在一起容易在功能需求中淹没非功能关注点
STR-F 按什么组织	用户角色 × 触发	按业务流程阶段 / 按业务模块 / 按需求来源部门	角色穷举 = 完备性可验证；其他方式做不到
STR-NFR 按什么组织	角色 → 一级类目 → 二级分类	只按分类不按角色 / 只按角色不按分类	角色保证相关方可验证，分类保证技术完备性——两面缺一不可
末级节点指代什么	STR-F：一个场景业务（端到端业务闭环），即一个完整的「某角色在某触发下做某事的端到端旅程」；STR-NFR：一个非功能特性维度	功能操作 / 业务模块	太细了下游 BP 的 N:1 合并爆炸，太粗了撑不起需求分配。判定标准：业务人员能否描述出一个完整的工作闭环？
冲突需求怎么处理	保留原样，标注冲突，等待裁决	预先合并 / 直接丢弃	STR 层不越界做方案设计决策；冲突裁决是相关方的权利

这一层的独特特征：STR 是设计链中第一个「方案」层——它向上承接 OR（N:1 全覆盖映射，可追溯验证），向下产出可传递的结构化需求体系。STR 的完备性由两条腿保证：向上追溯 OR 是否全覆盖（可追溯验证），向下靠角色集穷举（方法论保证——OR 本身可能没说全）。**STR 的每一个架构决策，本质上都在回答：「我们怎么知道需求说全了？」**

不可逆性：极高。不是因为改文档的成本高，而是因为它是整条链的根——根歪了，整棵树歪。

5.4 BP 层——定义「业务流程及其业务行为单元」

这一层在解决什么设计问题：将功能需求从「用户要做什么」（STR-F）转译为「产品如何支撑该业务——通过哪些输入→处理→输出的业务行为单元（IPO）」。BP 层定义的是业务流程的结构——按什么逻辑组织流程、每个流程节点对应一个什么样的业务行为单元。BP 不创造新的需求内容——全部输入来自 STR-F 的详细定义。它做的是转译：把用户要做什么（STR-F）转译为企业通过什么流程、产出什么业务行为单元。转译的质量取决于流程颗粒度和业务行为单元（IPO）颗粒度的判断。

BP 层最关键的决策——流程拆到什么颗粒度、每个节点是不是一个完整的 IPO：

流程颗粒度和 IPO 颗粒度是同一枚硬币的两面：一段流程对应一个「输入→处理→输出」的业务行为单元，流程拆多粗、IPO 就有多粗。以下用同一组例子展示两种视角的对应关系：

太粗了：
流程侧：「采购管理」——一个笼统的业务领域，没有具体的流程边界
IPO 侧：没有可识别的输入单元和输出单元
→ 这等于没分解。下游 SR 面对一个黑箱，不知道要支撑什么功能。

太细了：
流程侧：「填写供应商名称」——这不是一段独立流程，只是创建采购订单流程中的一个步骤
IPO 侧：没有独立的输入起点和输出终点，只是订单中的一个字段
→ 业务上没有人会说「我今天的工作是填写供应商名称」，它只是更大的业务行为中的一个动作。
→ STR 的端到端业务有 30 个，拆成 300 个步骤级节点会让 SR 层的 N:1 合并量爆炸。

刚好：
流程侧：「创建采购订单」——一段完整的、可独立被业务人员讨论的业务活动
IPO 侧：输入（收到请购需求）→ 处理（选定供应商、组装订单）→ 输出（下达采购订单）
→ 业务人员可以指着这段流程说：「对，我这一步就是创建采购订单，收到请购需求、产出采购订单。」
→ 其内部步骤（如填写供应商名称）属于处理（P）内部的详细展开，不在此层的架构决策范围内。

颗粒度的判定标准——遵循通用规范对 BP 的定义：**每个 IPO 必然以一个输入单元起始、以一个输出单元终止——**没有输出则不可记录、不可追溯，不构成工程意义上的业务行为：

- 能否识别出明确的**输入**（某类外部角色向产品提交了什么）？
- 能否识别出明确的**输出**（产品向用户角色交付了什么，且得到其确认）？

能 → 刚好。输入/输出模糊、一个节点覆盖多段不同输入输出的流程 → 太粗。只是某个 IPO 处理内部的一步、缺少独立的输入起点和输出终点 → 太细。

BP 层的形式化架构决策：

决策	内容	备选方案	为什么架构决策
顶层按什么组织	业务域 → IPO 两级 （业务域按业务职能领域划分——采购、生产、销售、售后……，每个业务域下直接聚合其所属的 IPO，无需中间分类层级）	按职能部门组织（需求来源视角） / 按系统模块组织（技术实现视角）	业务域是企业业务流程的自然聚类——业务流程跨部门运转，按域组织才能保持端到端流程的完整性。固定两级保证从业务全景到可承接系统功能点的直接收敛

决策	内容	备选方案	为什么架构决策
末级节点指代什么	业务行为单元 (IPO) ——某类外部角色向产品提交一类输入，产品经处理（可含人的专业工作）向用户角色产出满足其期望并得到确认的输出	停在端到端业务（太粗——SR 层无法确定要为哪些行为单元设计哪些功能）/ 拆到处理步骤（太细——SR 层 N:1 合并爆炸，几十个步骤级节点才对应一个系统功能点）	颗粒度决定 SR 层承接效率。判定标准：每个节点是否以唯一输入起始、以唯一输出终止
流程结构怎么推导	用户角色架构锚定法（锚点在角色集）	按现有业务流程文档梳理（依赖现状，无法保证完备性——你只能整理出已经有人写下来的流程）/ 按组织架构图推导（混淆了管理归属和业务逻辑——同一个流程可能跨三个部门）	锚点决定完备性保证方式。角色穷举保证覆盖所有使用者视角
要不要详细定义	BP 无独立详细定义子文档；每个 IPO 节点以通用四要素承载定义（定义、所映射的上层需求、符合性分析、节点关系），其中输入（I）、处理（P）、输出（O）在此层定义	像 STR 一样设独立详细定义子文档	BP 是桥——把功能需求转化为业务行为单元后直接交给 SR 做视角转换。处理（P）内部的详细步骤不在此层的架构决策范围内

流水线类产品（如 EOS）的特化：BP 的两级结构在流水线类产品中可特化为多级——业务域下增加「项目类型」（按输出产品类型切分）、「端到端业务」、「WBS（可选）」等中间层级；软件类产品的 IPO 输出形态常表现为「输出文档」；流程推导可用输出产品架构锚定法（锚点在输出产品 PA 节点类型）。该特化以产品级规范为准（见 EOS 产品规范），不属于通用方法论强制结构。

这一层的独特特征：BP 不创造新内容，它做的是**转译**——把「用户要做什么」转译为「产品通过什么业务行为单元（输入→处理→输出）支撑该业务」。转译的质量取决于 IPO 颗粒度的判断，而颗粒度判断严重依赖领域知识。

不可逆性：中-高。修改成本在文档层面，但 IPO 颗粒度决定了 SR 层桥接重组的起点——颗粒度错了，SR 层的功能分解就失去了正确的输入粒度。

5.5 SR 层——定义「系统功能的组织方式」

这一层在解决什么设计问题：将 BP 层的业务行为单元（IPO）转化为系统功能需求（SR-F），将分散的非功能关注点归并为统一的技术约束（SR-NFR）。这是设计链中**第一次出现「系统内部」概念**的层——在此之前，STR 和 BP 都在描述「系统外面有什么、业务上要产出什么业务行为」。SR 第一次问：「支撑这些业务行为单元的生成和管理，系统内部需要提供什么功能、满足什么约束？」

SR-F 的核心架构决策：

决策	内容	为什么架构决策
功能分类按什么组织	系统职责视角——功能按系统内部职责聚类，而非按业务部门聚类	决定了开发者看到的系统功能组织结构

决策	内容	为什么架构决策
末级节点指代什么	一个系统职责视角下的功能需求单元——可独立分配、独立验证的系统能力单元	决定了 PA 层构件划分的输入颗粒度
BP→SR 的映射方式	桥接重组（视角转换 + 分类重组），不是 1:1 分配	BP 的业务行为单元与 SR 的功能点可以是同一个东西的两种视角描述
功能归并规则	按系统能力域归并——同一个业务行为单元（及其输出）衍生的多个系统功能归入同一能力域节点；不同能力域的功能不共享节点	决定了系统功能的凝聚度和内聚性

流水线类产品（如 EOS）的特化：在 EOS 产品中，SR-F 末级具体化为功能表单/配置页面（biz 路径为功能表单/窗口标签页，eng 路径为配置页面——见 EOS 91 规范），本文不展开该特化。

视角转换示例：

BP 产出：一个业务行为单元（IPO）—输入「请购需求」，输出「采购订单」，业务人员定义其构成与规则
SR 产出：「订单管理功能」—支持表单录入、数据校验、库存预占、审批流驱动
（以系统职责和能力为中心，描述系统要提供什么功能来支撑这个业务行为单元的全生命周期）

这两个可以是同一个东西的两种视角：BP 的描述是给业务人员看的（「我这一步产出的是一张采购订单，里面有这些字段、这些规则」），SR 的描述是给系统设计者和开发者看的（「系统需要提供订单管理的这些能力」）。

SR-NFR 的核心架构决策：

决策	内容	为什么架构决策
STR-NFR → SR-NFR 的归并规则	同一二级分类下、不同角色来源的关注点合并为一个 SR-NFR 节点	从「按角色」切换到「按分类」——相关方验证视图 → 工程师实施视图
指标冲突的裁决规则	同一维度多个角色提了不同指标值 → 取最严格者	满足最严格的自动满足所有更宽松的
分类视图切换	按角色组织 → 按一级类目组织、按二级分类归并	NFR 路径的关键视角转换——保证相关方可验证的同时工程师可查全量约束

这一层的独特特征：SR 层是设计链中第一次出现「系统内部」概念。STR 和 BP 都在描述系统外面有什么、业务上产出什么文档；SR 第一次描述系统内部要提供什么能力来支撑这些文档的全生命周期。

不可逆性：中。修改成本在文档层面，但 SR 的功能分类结构直接决定 PA 的构件划分起点。

5.6 PA 层——定义「构件的边界和契约」

这一层在解决什么设计问题：将系统功能分配和归并到可实现的产品构件中，定义构件之间的接口契约。PA 层不做新的功能设计——所有功能在 SR 层已经定义清楚。PA 层做的是**分配和归并**：100 个功能点，归并到 12 个构件里。决定功能点 A 和功能点 B 放在同一个构件还是不同构件——这就是 PA 层的架构决策。

这个决定有三个维度的约束： - **技术合理性**：A 和 B 共享数据吗？操作同一个数据实体 → 倾向合并 - **组织可行性**：A 和 B 可以由同一个团队维护吗？需要不同技能 → 倾向拆分 - **演化独立性**：A 和 B 会以同样的节奏变更吗？A 频繁变、B 稳定 → 倾向拆分

PA 层的形式化架构决策：

决策	内容	为什么架构决策
构件怎么划分	三独立原则——可独立分配、独立替换、独立验证	构件边界是整个设计链中最不可逆的架构决策
构件边界划在哪	信息处理模式硬边界 + 业务域边界 + 数据所有权边界三者取交集	边界划错了 → 数据迁移 + 接口重定义 + 团队重组
构件之间怎么交互	接口契约——同步/异步、数据格式、超时/重试/降级策略	接口契约是构件之间的法律——一旦对外发布，修改就是破坏性变更
构件从哪来	资产优先——复用→改进（扩展/拆分/合并）→新增	决定了架构决策的知识积累方式
NFR 怎么作用到构件	NFR 不分配到特定构件，作为 PA 整体约束	质量视图独立于功能分配，保证质量要求不被功能分解碎片化

PA 层最关键的一个决策——构件边界。为什么它是最不可逆的？

改了 STR 的分类维度 → 下游四层重做。但 STR 本身不改代码，改的是文档。

改了 IPO 的颗粒度 → SR 重新桥接。改的是文档。

改了 SR 的功能分类 → PA 重新映射。改的是文档。

改了 PA 的构件边界 → 不是改文档。

数据库要拆 → 数据迁移、双写过渡、历史数据兼容。

接口要重新定义 → 所有下游服务适配。

团队要重新划分 → 人员调动、知识交接、沟通链路重建。

不可逆性：极高。前三层的修改成本在文档层面，PA 层的修改成本在代码 + 数据 + 组织层面——这是量级的差异。

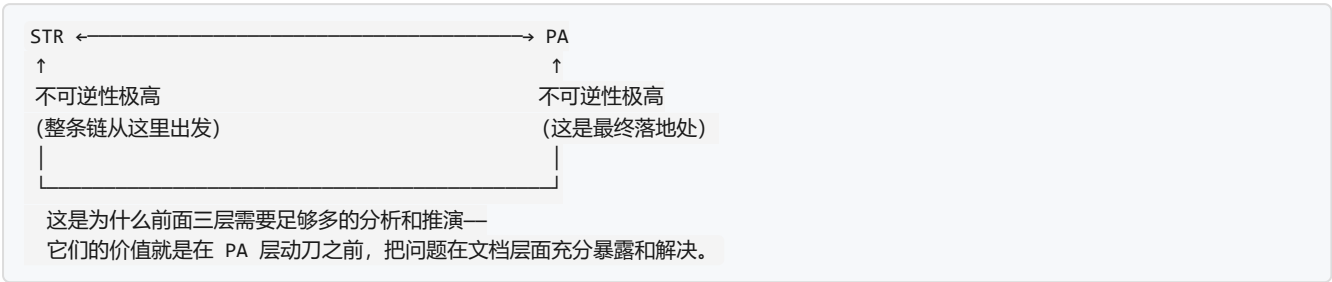
5.7 四层决策全景对比

	STR —	BP —	SR —	PA —
解决什么问题	需求怎么组织？	业务流程如何组织、产出什么业务行为？	系统功能和约束？	谁来干？
核心操作	分类 + 完备性保证	流程分解 + IPO 定义	视角转换 + 归并	归并 + 分配
末级节点	端到端业务闭环	业务行为单元（IPO）	系统功能点	产品构件
新增了什么？	组织维度 + 完备性判据	流程组织维度 + IPO 颗粒度判断	系统内部能力定义	构件边界 + 接口契约
修改成本	文档级	文档级	文档级	代码/数据/组织级
核心相关方	需求决策者	业务相关方	系统设计者	开发实施者
决策失误后果	整条链偏	流程结构不合理/ IPO 划分失当	功能混乱/NFR遗漏	系统无法稳定演化
不可逆性	极高	中-高	中	极高

注：上表 BP 列为通用基线（业务域 → IPO 两级）。流水线类产品（如 EOS）将其特化为多级

(项目类型/端到端业务/WBS/输出文档)，见 §5.4 注。

一个关键规律：



链的两端承受最高风险——STR 层的决策错了，后面四层全错；PA 层的决策错了，前面三层的设计在落地时需要返工。**中间两层（BP、SR）的决策修改成本在文档层面，它们的作用正是缓冲和吸收不确定性——在文档层面把问题充分暴露，避免带到代码层面。**

六、架构视图

本章从原理 2.4（视图完备性由相关方决定）出发，展开讨论如何系统性地推导一个系统需要哪些架构视图。

6.1 主流框架对比——为什么没有标准答案

历史上几乎所有架构框架都试图回答「需要几个视图」，它们的答案各不相同：

框架	几个视图	是哪几个	面向什么场景
Kruchten 4+1 (1995)	5	逻辑 + 进程 + 开发 + 物理 + 场景 (+1)	单个软件系统
Rozanski & Woods (2005)	7	上下文 + 功能 + 信息 + 并发 + 开发 + 部署 + 运维	单个软件系统 (比 4+1 更细致)
TOGAF (最新版)	4A	业务 + 应用 + 数据 + 技术	企业级 IT 资产管理
DoDAF (美国国防部)	8 类 52 个	全视点 / 能力 / 数据与信息 / 作战 / 项目 / 服务 / 标准 / 系统	巨型系统体系 (武器+指挥+后勤.....)
ISO 42010 (2011)	不定	不规定——只规定「怎么从相关方推导视图」	通用

每个框架的数字不一样，而且跨度极大（4 到 52），但没有一个框架说「6 个」或者「8 个」是标准答案。这不是因为某个框架对、某个框架错，而是因为每个框架面向不同类型的系统、不同类型的企业、不同类型的相关方。DoDAF 有 52 个视图是因为它面对的是美国国防部的相关方离散度——国会（出资方）、各军种（使用者）、承包商（构建者）、测试评估机构（鉴证者）、盟军（集成方）——每一方都需要用自己能理解的语言看到关心的内容。

正确的思维方式不是「挑一个框架的视图清单来用」，而是**从自己的相关方出发重新推导**。

6.2 推导方法——从相关方到视图的三步法

第一步：识别相关方及其关注点

这是绝对不能跳过的一步。相关方是谁，不是方法论告诉你的——是你通过分析具体项目得出来的。

但有五类相关方由「系统从概念到运行（含退役处置）的完整生命周期」决定——退役阶段的相关方关注点归入运维者（退役执行）和鉴证者（退役合规验证），不单独设立相关方类别。这五类相关方**几乎必然出现**——不是因为方法论规定了它们，而是因为系统存在的逻辑本身就隐含了这些相关方：

相关方类别	为什么必然存在	核心关注点
采办者 (Acquirer)	有人出钱/出资源让系统存在	系统能解决我的问题吗？花多少钱、多少时间？
使用者 (User)	系统存在的唯一目的是被使用	我怎么用？能帮我做什么？好用吗？
构建者 (Builder)	抽象设计不会自己变成运行的系统	我要做什么？边界在哪？依赖什么？
运维者 (Operator)	任何系统运行就需要被运维	怎么部署？怎么监控？故障如何恢复？容量够不够？
鉴证者 (Assurer)	系统存在于一个有约束的环境中（法规/标准/合规/安全）	安全吗？合规吗？审计能过吗？违反法规谁担责？

关于“鉴证者”——“鉴证”= 先**鉴**（独立审视系统是否满足外部标准），后**证**（给出认定结论）。这个角色不建造、不运维、不使用——只做检查和认定。消防检查官就是典型的鉴证者。

这五类相关方不是「选择的」，是「识别出来的」。你可能在某些项目里两类合并成一个人（比如构建者也运维），但关注点本身不会消失。

为什么没有“物料供应方”？

五类的划分逻辑是**生命周期角色**——每个角色对应系统从概念到退役的一个必然阶段：被采办（采办者）→ 被设计/建造（构建者）→ 被使用（使用者）→ 被运维（运维者）→ 被鉴证（鉴证者）。物料供应方不在这条生命周期链上——它在**供应链**上，是构建者的上游，对系统的关切是“我的产品被正确使用”而非“这个系统的架构是否合理”。

但物料供应方作为一个**条件性相关方**是成立的：当系统严重依赖某个特定外部组件——专有协议、独家硬件、特定云服务——且该组件的演进方向会制约架构选择时，供应方应纳入相关方分析。关键在于：不是“所有供应商都算”，而是“供应商的决策会影响你的架构决策时，才算”。

第二步：将关注点聚类为视图

每个相关方有一堆关注点。但**不是每个关注点都需要一个独立的视图**——那样视图数量会爆炸。需要按两个原则聚类：

原则 1：同一视图内的关注点，可以用同一套语言、同一种表达方式来回答。比如使用者的「我怎么做采购下单？」和采办者的「系统支持了哪些业务流程？」——都可以用业务场景叙事来回答，可以放在

同一个视图里。但使用者的「我怎么做采购下单？」和运维者的「数据库怎么备份？」——需要完全不同的语言，必须放在不同视图里。

原则 2：视图的数量上限，由人类同时能有效处理的视角数量决定。如果一个架构描述有 15 个视图，没人能看完——它**实际上不会被人使用**，等于不存在。经验上：3-5 个视图绝大多数人能有效理解和维护；6-8 个需要架构团队专业维护，相关方按需查阅；9+ 个通常意味着聚类不够，存在冗余。

第三步：检查完备性

聚类之后，用两个问题自检：

1. 有没有重要相关方的核心关注点，在任何一个视图中都找不到答案？

→ 如果有，缺视图，需要增加或扩展现有视图。

2. 有没有视图的内容，不被任何重要相关方需要？

→ 如果有，这个视图是多余的，删掉或合并。

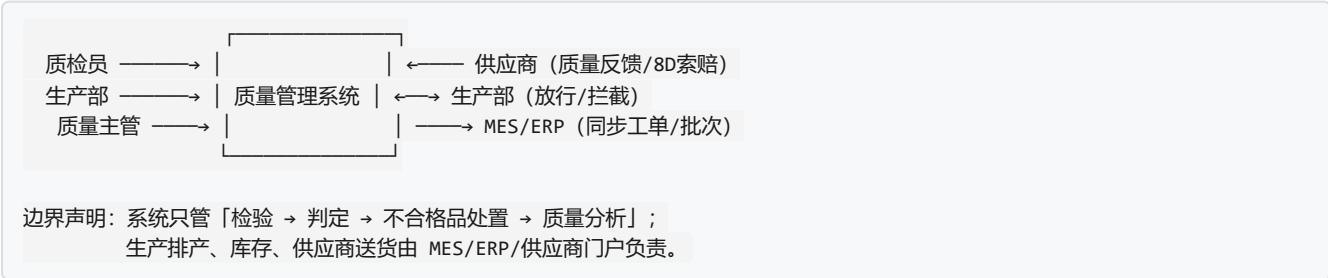
6.3 七候选视图

用三步法完整走一遍，对于绝大多数业务系统/软件产品，答案会收敛到七个候选视图。为便于理解，以下 7 个候选视图以同一系统——**质量管理业务系统**（QMS）——为例（贯穿场景：一批来料检验 → 发现不合格 → 不合格品评审处置 → 反馈供应商 → 质量分析）。同一个系统、七个视角，正呼应 §1.3 的骨骼比喻——同一副骨骼，从不同角度拍的片子：

候选 1：系统上下文视图（Context View）

维度	内容
回答什么	系统在哪？边界在哪？和外界是什么关系？
目标相关方	所有相关方（这是理解系统的起点）
核心表达	系统边界图 + 外部实体清单（用户、外部系统、环境）+ 交互关系
可省略性	几乎总是需要。没有边界就没有系统

示例：画一张「系统边界图」——中间是系统，外面是与它交互的人和系统，箭头标出交互，图外一句话圈定范围：



一张图同时回答「谁在外面、怎么连的、什么不管」。

候选 2：用户操作视图（User Operational View）

维度	内容
回答什么	用户怎么使用系统来完成他们的工作？哪些角色、在什么场景、执行什么操作？
目标相关方	使用者、采办者
核心表达	用户角色与职责 + 操作场景（用户故事/用例）+ 用户端到端工作流 + 用户与系统的交互序列
可省略性	几乎总是需要。采办者通过它验证「系统能不能支撑业务」，使用者通过它确认「我的工作怎么做」

为什么需要一个独立视图？ 系统上下文视图说清了「谁在外面、怎么连着」，但不回答「外面的这些人具体怎么用系统做他们的事」；功能/行为视图的主语是系统（系统提供什么、系统内部怎么协作），不回答用户的操作流程。用户操作视图填补了这个空白——主语是**用户**，描述的是运行概念（Operational Concept）：用户按照什么流程、在什么场景下操作系统来达成自己的工作目标。

示例：列「角色 × 场景」的操作概念结构——主语是用户（角色），描述每个角色在什么场景、按什么端到端流程操作：

角色 × 场景（操作概念的结构，不是某一天的流水账）：
质检员 × 来料检验：取任务 → 抽样 → 录入读数 → 判定（放行/拦截）
质量主管 × 不合格处置：接 NC 评审单 → 会签评审 → 处置（让步/返工/退货/报废）
供应商 × 质量反馈：收整改要求 → 提交整改 → 关闭

以「来料检验」为例的端到端流程结构：
报检 → 抽样方案匹配 → 检验记录 → 判定 → 合格放行 / 不合格转 NC 评审

采办者用这个视图验证「系统能不能支撑质检业务」，质检员用这个视图确认「我的工作怎么做」。

候选 3：功能/行为视图（Functional/Behavioral View）

维度	内容
回答什么	系统提供哪些功能？这些功能之间如何动态协作？
目标相关方	构建者、使用者
核心表达	功能分解（功能树/功能分类）+ 功能间的动态交互（顺序、状态、活动）+ 功能与需求的映射
可省略性	几乎总是需要。系统对绝大多数相关方来说首先是「能做什么」

为什么不是“用户怎么使用它”？ “用户怎么使用”的主语是用户，描述的是用户的操作流程——那属于运行概念，应在用户操作视图中描述（见候选 2）。功能/行为的主语是**系统**——功能 = 系统的静态能力分解（系统有什么能力），行为 = 系统的动态协作模式（这些能力之间怎么一起工作）。

示例：画两样东西——一张功能分解树（静态：系统有什么能力）+ 一条状态机/协作序列（动态：这些能力怎么一起干活）：

功能分解（静态）：
质量管理系统
├─ 检验管理（来料 IQC / 过程 IPQC / 成品 FQC，抽样方案配置）
├─ 不合格品管理（NC 单、评审会签、处置执行）
├─ 供应商质量（质量反馈/8D、供应商绩效评分）
└─ 质量分析（SPC 控制图、帕累托、批次合格率报表）

不合格品单状态机（动态）：
待评审 → 提交 → 评审中 → 会签通过 → 已处置（让步接收 / 返工 / 退货 / 报废）
└─退回→ 待补充资料（重新提交）
处置「退货」 → 触发供应商质量反馈单；处置「返工」 → 通知生产部开返工工单

与候选 2 的差异：这里主语换成了系统——「不合格品服务支持评审流转」，而不是「主管去评审」。

候选 4：信息/数据视图（Information/Data View）

维度	内容
回答什么	系统管理什么数据？数据怎么组织、归谁管、怎么流动？
目标相关方	构建者、鉴证者（数据合规）
核心表达	领域数据模型 + 数据所有权 + 数据流
可省略性	系统的数据很简单、不共享、不需要建模时可合并到其他视图

示例：列核心数据对象 + 数据所有权 + 一条关键数据流——停在概念层，不落到表结构与基数：

核心数据对象：检验批次、检验记录、物料（主数据）、不合格品单（NC）、处置记录、供应商质量档案

数据所有权：检验/NC/质量分析数据归「质量域」自有库；
物料主数据归「主数据域」；批次/工单溯源归 MES，域间仅经接口取数。

数据流（概念）：检验判不合格 → 生成 NC 单 → 评审会签 → 处置结果回写批次 → 触发供应商反馈/生产放行

「数据归谁管、谁在用、怎么流动」三问，一张图全有。

候选 5：构建/开发视图（Construction/Development View）

维度	内容
回答什么	系统由哪些构件组成？构件之间怎么交互？
目标相关方	构建者（开发者）
核心表达	构件清单 + 构件定义 + 构件接口契约 + 构件依赖拓扑
可省略性	几乎总是需要。这是开发者唯一能看到「自己该做什么」的视图

示例：列构件清单 + 构件间契约（语义层） + 依赖拓扑——契约只到「职责 + 同步/异步 + 跨边界传什么」，不落到 HTTP 语法：

构件清单：
检验服务（检验任务、抽样、检验记录）
不合格品服务（NC 单、评审流、处置）
质量分析服务（SPC、帕累托、报表）
质检工作台（前端，含平板端）

构件间契约（语义层）：
检验服务 → 同步 → 不合格品服务 传递「批次号 + 不合格项 + 判定依据」，发起 NC 评审
不合格品服务 → 异步 → 检验服务 处置结果回写检验批次状态
不合格品服务 → 异步 → 供应商（外部） 处置为「退货」时触发质量反馈/8D 单
（契约约束：跨构件调用须定义同步/异步与传递的数据语义；对外契约变更属破坏性变更）

依赖拓扑：质检工作台 → 检验服务 → 不合格品服务 / 质量分析服务

构件边界 + 职责契约 = 开发拆活干活的依据，也是 §5.6「构件边界是最不可逆决策」的落点。

候选 6：部署/运行视图（Deployment/Runtime View）

维度	内容
回答什么	系统跑在什么环境上？怎么部署、监控和维护？
目标相关方	运维者、构建者
核心表达	部署拓扑 + 基础设施需求 + 运行时依赖 + 监控/告警策略
可省略性	单机简单系统可合并到构建视图；分布式/云原生系统必须独立

示例：画部署形态 + 监控维度 + 架构约束——物理层只到「部署在哪里、什么形态」，容量实例（几副本、几主几从）与告警阈值不属于架构视图：

部署形态（物理层）：
 质检工作台 → 云端 CDN（车间平板/浏览器访问）
 检验/不合格品服务 → 内网集群（产线门禁：低延迟、可独立扩容）
 质量分析服务 → 独立部署（批处理出日报/月报）
 数据库 → 主从 + 每日备份（检验记录须留存）

监控维度：待检批次积压、检验服务可用性、NC 单处理时长（均为产线门禁指标）

架构约束：① 独立部署/独立扩容 ② 产线时段可用性目标（停机即停线）③ 检验记录可追溯留存

与候选 5 的区别：候选 5 关心「有哪几个构件」，这里关心「构件跑在什么环境、怎么保证不挡产线」。

候选 7：质量/约束视图（Quality/Constraint View）

维度	内容
回答什么	系统要满足什么非功能约束？有哪些合规要求？
目标相关方	鉴证者、运维者、采办者
核心表达	NFR 分类清单 + 质量指标 + 合规要求 + 技术约束
可省略性	至少需要嵌入到其他视图中；质量要求复杂的系统（安全/性能/可用性/合规各自有独立相关方）必须独立

示例：列一张 NFR 承诺清单——鉴证者在这里一次性看到全部非功能承诺：

性能 : 检验记录提交 p95 < 1s；扫码取批响应 < 300ms（车间平板）
可用性 : 产线运行时段可用性 ≥ 99.9%—质检是放行门禁，停机即停产
安全 : 检验记录防篡改、全程留痕；质检员不可改判定规则与抽样方案（按角色授权）
合规 : 检验记录保留 3 年且可追溯；符合 ISO 9001；计量器具校准记录留档
可维护性 : 抽样方案/判定规则可配置（改规则不动代码）；构件独立部署、独立替换

审计者不必翻遍功能文档找质量承诺——这张图就是「系统在质量上承诺了什么」的单一事实源。

6.4 相关方 × 视图对应关系

一个相关方通常需要不止一个视图：

相关方	主要视图	辅助视图	备注
采办者	系统上下文 + 用户操作	功能/行为 + 质量/约束	采办者需要知道「系统能支撑哪些业务操作」和「系统边界在哪」
使用者	用户操作	系统上下文 + 功能/行为	使用者首要关心「我的工作怎么做」，其次才是系统能做什么事
构建者	构建/开发 + 信息/数据	部署/运行 + 功能/行为	—
运维者	部署/运行 + 质量/约束	构建/开发	—
鉴证者	质量/约束	系统上下文 + 信息/数据	鉴证者需要一个地方一次性看到全部 NFR 承诺

6.5 决定视图数量的三个因素

因素	对视图数量的影响
系统复杂度	简单系统（单机工具、静态网站）3-4 个视图就够，部署和质量可合并到构建视图；中等系统（企业应用、SaaS）5-6 个视图；复杂系统（分布式平台）7 个视图且某些视图需细分——部署可能拆为运行时进程视图和物理部署视图，数据可能拆为概念数据模型和物理数据存储，质量必须独立
相关方离散度	所有相关方同属一个组织 → 视图可少，一门语言沟通；跨部门 → 每个部门至少需要一个看得懂的视图；跨公司（甲乙方、多方集成）→ 每个外部相关方可能需要独立视图
生命周期阶段	概念阶段：上下文 + 功能（粗粒度）即可；设计阶段：需要全部但各视图详细程度不同；构建阶段：构建 + 部署最详细；运维阶段：部署 + 质量最活跃

6.6 完备性自检——不看数量看覆盖率

自检问题	对应视图
一个完全不懂技术的人，能不能看懂系统的边界和目的？	系统上下文视图
一个业务用户，能不能搞清楚自己在系统里的工作流程和操作场景？	用户操作视图
一个开发者，能不能搞清楚系统提供哪些功能、功能之间如何分类和协作？	功能/行为视图
一个新加入的开发者，能不能搞清楚系统有哪些构件、每个构件管什么、构件之间怎么交互？	构建/开发视图
系统管理和使用了哪些核心数据？数据归谁管、谁在用？	信息/数据视图
一个运维工程师，能不能知道系统跑在哪里、怎么部署、怎么监控？	部署/运行视图
一个安全审计者，能不能一次性看到系统的全部 NFR 承诺和合规要求？	质量/约束视图

如果上面某个问题的答案是「不能」——你的视图不够，不管你现在有几个。

6.7 追溯——贯穿全部视图的横切面

严格来说它不是第八个视图，而是贯穿全部七个视图的一条线——可追溯性。

任何一个相关方在任何一个视图中看到任何一个元素，都应该能回答：「这个东西是从哪里来的？它要满足谁的需求？」

追溯的目标相关方是**所有相关方**：业务相关方看到业务操作上的性能指标 → 追溯到需求来源 → 确认「这确实是我提的」；开发实施者看到构件上的接口定义 → 追溯到功能需求 → 确认「这个接口是为了满足那个功能」；系统设计者收到需求变更 → 追溯变更影响的全部构件 → 评估变更规模。

追溯不独立成为一个视图，但它是**所有视图之间的连接组织**——没有它，七个视图就是七张互不相关的图。

6.8 视图 vs 文档——一个关键区分

视图 ≠ 文档。一个视图可能由多个文档承载，一个文档也可能服务于多个视图。

视图是**逻辑概念**——它定义了「谁看什么」。文档是**物理概念**——它是承载内容的文件。二者的关系是 N:M：

- 系统上下文视图可能由「系统概述」一个文档承载，也可能由「相关方需求」「系统边界定义」「外部接口规范」三个文档共同承载
- 一个「产品架构文档」可能同时服务于构建视图（构件清单部分）和部署视图（部署拓扑部分）

混乱的根源往往是：**把物理文档的数量当成了逻辑视图的数量，或者反过来，要求每个逻辑视图都对应一个独立文档。**

6.9 视图与设计链的映射——以一个五层链为例

把七候选视图映射到一个具体的五层设计链（OR → STR → BP → SR → PA），可以看到每层产出对应了哪些视图：

七视图	对应的设计链产出	覆盖状态
系统上下文视图	OR + STR（系统边界、角色集、外部交互）	✔ 覆盖
用户操作视图	STR（用户角色）+ BP（业务场景、端到端 workflow）	✔ 覆盖——用户视角的操作流程贯穿 STR 角色定义和 BP 业务场景
功能/行为视图	STR-F + BP + SR-F（需求→业务行为单元→系统功能）	✔ 覆盖，但分散在三层
信息/数据视图	BP（业务数据流）+ SR/PA（逻辑/物理数据模型）	⚠ 部分覆盖——数据架构的视角分散在多层，无独立聚合点
构建/开发视图	PA（构件清单 + 构件定义 + 接口契约）	⚠ 大部分覆盖——构件关系（接口契约、依赖拓扑）的显性化程度不一

七视图	对应的设计链产出	覆盖状态
部署/运行视图	—	✗ 未显式覆盖——部署拓扑、基础设施需求、运行时约束在设计链中不在设计范围内
质量/约束视图	STR-NFR + SR-NFR (NFR 分类 + 指标)	✓ 覆盖——NFR 有独立平行路径，不与功能混淆
(追溯)	追溯矩阵	✓ 覆盖——贯穿全部视图的连接组织

两个值得关注的缺口：

缺口 1：部署运维视图的缺失。 设计链止于 PA 层，部署属于集成阶段，运维属于运维阶段。这本身没有问题——部署和运维的详细内容确实不属于「系统设计」。但系统设计阶段需要知道**部署运维对架构的约束**——比如「这个构件必须能独立部署」「这个数据必须支持跨数据中心同步」——这些约束如果不在系统设计阶段提出来，到了集成/运维阶段再发现就会变成架构返工。建议在设计终点（PA 层）增加一个轻量的「部署运维架构约束」章节——不是完整部署手册，只需要回答：系统设计中有哪些决策是因为部署/运维约束而做出的。

缺口 2：构件关系的显性化程度。 构件之间的关系（接口契约、调用拓扑、数据流）往往比单个构件的定义更重要——因为系统的复杂性不在构件内部，而在构件之间。当构件间关系足够复杂时，可以将其从附属说明提升为架构定义的一个独立部分，如「构件关系视图」（包含接口契约矩阵 + 构件依赖拓扑）。

七、企业架构4A模型——为什么是这四个？

有了前六章的基础，本章专门回答一个困扰很多人的问题：**为什么企业架构（EA）翻来覆去就是「业务架构、应用架构、数据架构、技术架构」这四个？它们和架构视图是什么关系？**

7.1 4A的起源——不是设计出来的，是长出来的

TOGAF 的 4A（BDAT）可以追溯到 1987 年 John Zachman 的框架，后来被 TOGAF 吸收并固化。它的形成逻辑不是「相关方需要什么视图」，而是「企业里有什么东西需要被管理」：

企业有什么？ → 有业务流程和业务能力 → 叫「业务架构」
→ 有应用系统 → 叫「应用架构」
→ 有数据 → 叫「数据架构」
→ 有技术基础设施 → 叫「技术架构」

这是按管理对象分类，不是按相关方关注点分类。

7.2 4A就是四个管理地盘

从 1987 年到现在，这四分类几乎没变过——因为它刚好映射到大型企业的组织架构：

EA 域	对应组织中哪个部门	谁在管
业务架构	业务部门、流程管理部	业务负责人

EA 域	对应组织中哪个部门	谁在管
应用架构	IT 应用管理部	应用架构师
数据架构	数据管理部	数据架构师
技术架构	IT 基础设施部	基础设施架构师

4A 就是四个管理地盘。 4A模型本质上是一个**资产管理框架**——它回答「企业有哪些 IT 资产、每个资产归谁管」。而架构视图回答的是「这个系统的架构怎么让所有重要相关方理解」。两者服务于不同的目的。

7.3 4A × 七视图——两个分类维度的交叉

4A（按管什么切）和七视图（按给谁看切）是两个独立的分类维度，它们可以交叉：

	业务域	应用域	数据域	技术域
系统上下文视图	✔ 边界定义	✔ 系统交互	-	-
用户操作视图	✔ 核心覆盖	✔ 用户交互	-	-
功能/行为视图	✔ 核心覆盖	✔ 应用功能	✔ 业务数据流	-
信息/数据视图	-	✔ 数据模型	✔ 核心覆盖	-
构建/开发视图	-	✔ 核心覆盖	✔ 物理存储	✔ 技术选型
部署/运行视图	-	✔ 应用部署	✔ 数据部署	✔ 核心覆盖
质量/约束视图	✔ 业务SLA	✔ 应用性能	✔ 数据质量	✔ 技术SLA

4A是纵向切——把架构描述按管理归属分类。七视图是横向切——每个视图可能横跨 4A 的多个分类。

7.4 4A模型的三个问题

单独使用4A作为架构描述框架，存在三个结构性缺陷：

问题 1：需求视图被「业务架构」吞掉了。 在 TOGAF 里，需求管理是一个中心圆环放在 4A 的中间，但需求本身不被当成一等视图来对待。这导致需求只在项目初期被认真对待，然后就变成了一堆散落的文档，没有人持续维护它们和下游设计的关联。

问题 2：质量/NFR 散落在 4A 里，没人对它负责。 安全性在哪？在技术架构里（安全基础设施），也在应用架构里（应用安全），也在数据架构里（数据脱敏、权限），也在业务架构里（合规流程）。结果：安全性被 4A 各分类瓜分，没有一个视图能让相关方一次性看到「这个系统在安全上到底做了什么承诺」。质量要求需要一个独立的一等视图——不是 4A 各分类各自带一点。

问题 3：4A 各分类之间的动态关系丢失了。 4A 各分类各自是静态的快照。但架构中真正重要的东西——需求的变迁如何影响设计、业务场景如何转化为系统功能、一个构件的修改如何波及上下游——这些跨分类的动态关系在 4A 模型中没有被显式表达。

7.5 4A为什么这么流行

尽管有上述缺陷，4A模型依然占据统治地位。三个原因：

原因 1：TOGAF 认证体系锁定了话语权。 全球几十万 TOGAF 认证者，教材怎么写他们就怎么讲，考试怎么考他们就怎么信。

原因 2：它确实够用——在特定场景下。 当你问「公司有多少个应用系统？它们用哪些数据库？跑在什么服务器上？」——4A模型完胜。它是一个优秀的 IT 资产管理工具。但当你问「这个系统的架构是什么」——需要让业务方、需求方、设计者、开发者、运维者都理解系统——4A模型捉襟见肘。

原因 3：简化的力量。 四个概念，一张 PPT 放得下。业务/应用/数据/技术——谁都觉得自己能理解，不需要额外学习。你讲「七视图」需要解释每个是什么，你讲「4A」对方点头说懂了（虽然大概率理解不一致）。

7.6 4A与视图的共存——各司其职

	4A（BDAT）	七候选视图
目的	企业级 IT 资产的 管理和治理	一个系统架构的 理解和沟通
组织逻辑	按管理对象分类（谁管这个）	按相关方关注点分类（谁关心这个）
粒度	企业级（覆盖所有系统）	系统级（聚焦一个系统）
适用场景	IT 资产管理、技术路线规划、跨系统整合	系统设计、实现、部署、运维
谁在用	EA 团队、CIO、IT 治理委员会	系统设计者、开发团队、业务相关方

4A是「管资产」的框架。七视图是「描述系统」的框架。一个企业可能有 200 个系统——4A帮你说清楚这 200 个系统的资产分布。但当你聚焦其中 1 个系统做设计时，七视图帮你说清楚这个系统的架构让所有人理解。

八、业务架构实践

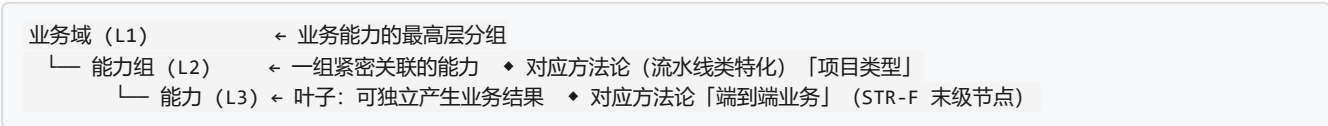
前七章讨论了架构是什么、架构决策怎么做、设计链中各层的架构决策有何不同——这些都是从**方法论和概念**的角度展开。本章换一个视角：从**企业实践**的角度，讨论业务架构（BA）长什么样、怎么描述、有什么价值、怎么让相关方认可、以及如何用它系统性提升业务能力成熟度。

8.1 业务架构视图的形态

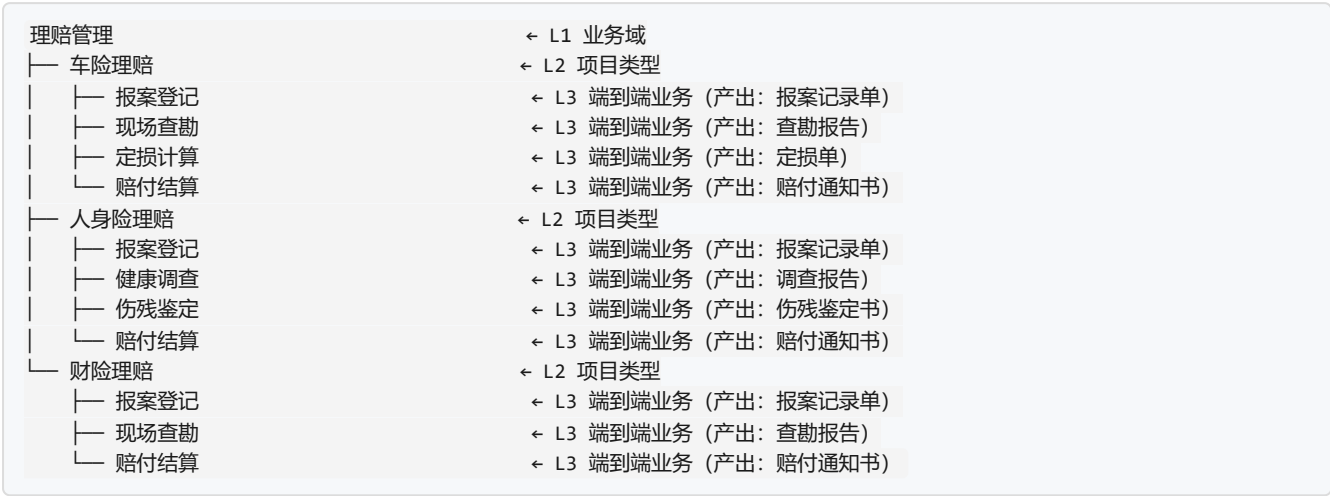
BA 视图在行业实践中最常见的形态是**分层的能力地图**（capability map）。它不是流程图、不是组织架构图、不是系统功能清单——它是一个企业“能做什么”的结构化目录。

称谓说明：本章描述行业 BA 的能力地图实践。能力地图的多级称谓（能力组/项目类型/端到端业务/输出文档）取自行业实践与流水线类产品（如 EOS）的 BP 特化；通用方法论的 BP 为「业务域 → IPO」两级（\$5.4）。本章为说明原理沿用多级称谓，通用口径以 \$5.4 为准。

8.1.1 基本结构



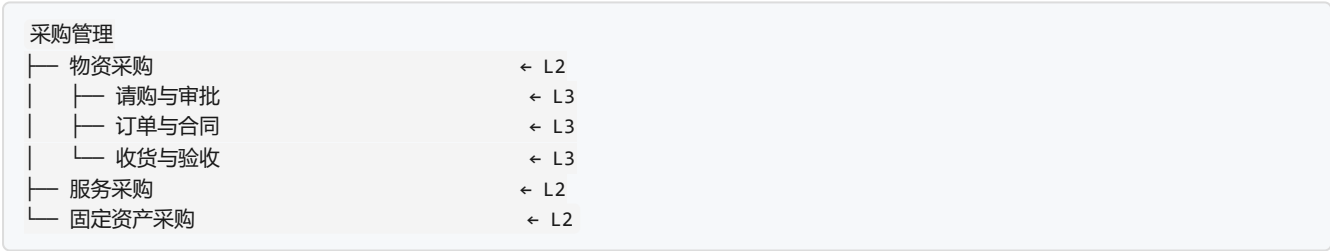
以保险公司理赔域为例。注意 L2 是按输出产品类型切分的**项目类型**（车险理赔 / 人身险理赔 / 财险理赔）——项目类型由该项目所输出产品的构件类型决定，不是理赔的生命周期阶段（报案→查勘→理算→赔付，后者是流程，不是类型）：



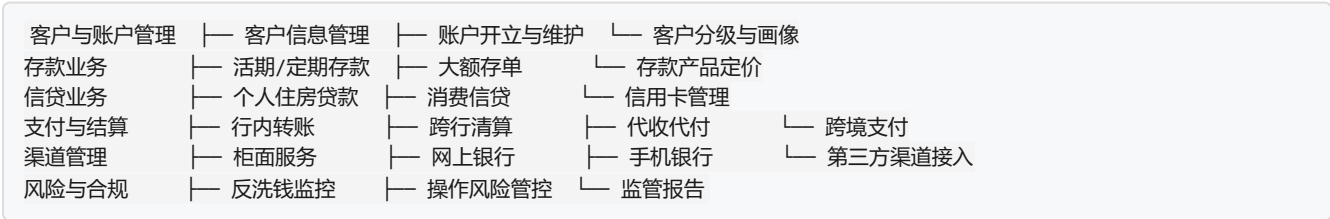
同一个端到端业务（如“报案登记”“赔付结算”）可以复现在不同项目类型下——这是因为不同险种的报案流程和赔付规则不同，它们是**同名但不同的构件类型**，各自有独立的输出文档和业务规则。

表达上通常是三层、最多四层。超过四层要么是混入了流程细节，要么是把不同视角的东西塞进了同一棵树。

以采购域为例，展示 L2 和 L3 的展开：



一个完整企业的 BA 顶层视图（零售银行举例）：



一家中型银行大概 6-8 个 L1 业务域，每域 3-8 个 L2，每 L2 再拆 3-8 个 L3 端到端业务——加起来大几百个端到端业务。这张图的价值不在于“画对了没有”，而在于它成了全企业的**共同语言原点**：业务部门用它描述自己要什么，IT 部门用它定位系统支撑范围，合规部门用它划定管控边界，管理层用它评估能力差距和投资优先级。

8.1.2 什么不是业务架构视图

三种常见的混淆：

混淆对象	为什么不是 BA	示例
流程图	描述的是"先做什么后做什么", BA 描述的是"能做什么"	"接报案→派工→查勘→定损→核赔→赔付"——这是流程, 不是能力
系统功能清单	主语是系统(系统能做什么), BA 的主语是企业(企业能做什么)	"OCR识别查勘单"——这是系统功能, 它支撑了"现场查勘"这个业务能力, 但它本身不是业务能力
组织架构图	描述的是"谁在管", BA 描述的是"管什么"	"理赔一部负责车险理赔"——这是组织归属, 不是能力定义

判定标准很简单: **如果明天换了一套系统、调整了组织结构, 这个东西还在吗?** 在的——是业务能力。不在了——是流程步骤、系统功能或组织单元。

8.2 业务能力的颗粒度

8.2.1 停在"能独立产生业务结果"这一层

行业共识(BIZBOK、TOGAF 业务架构实践)中, BA 的叶子止于 L3 (对应本方法论的"端到端业务"), 不继续往下拆:

层	叫什么	例子	判定标准
L1	业务域	理赔管理	业务能力的最高层分组
L2	项目类型	车险理赔	一组紧密关联的端到端业务
L3	端到端业务 (对应本方法论 STR-F 末级节点)	定损计算	能独立产生一个业务上可识别、可讨论、可度量的结果
L4+	WBS/流程步骤	零件工时计算	✗ 不放在 BA 里——属于流程设计的范畴

核心判断就一条: **业务人员能否指着这个节点说「对, 这件事是我们业务的一部分, 它有明确的产出、可以独立讨论和改进」?**

- "定损计算" → 能。输入是查勘信息, 输出是定损金额。业务人员可以独立讨论"定损规则对不对""定损时效达不达标"。
- "零件工时计算" → 不能。它只是定损计算的一个中间步骤, 不独立产生业务结果。把它放在 BA 里, BA 就会膨胀成一本流程手册。

8.2.2 层级混乱的典型表现

把流程步骤和端到端业务平级放置, 是最常见的错误:

✗

— 车险理赔

— 查勘调度

— 现场查勘

— 照片上传

— 定损计算

— 核价

← 这是"现场查勘"的信息采集步骤, 不是端到端业务

← 这是"定损计算"的一个环节, 不是端到端业务

修正后：

✓

车险理赔

├ 查勘调度

├ 现场查勘

└ 定损计算

← L2 项目类型

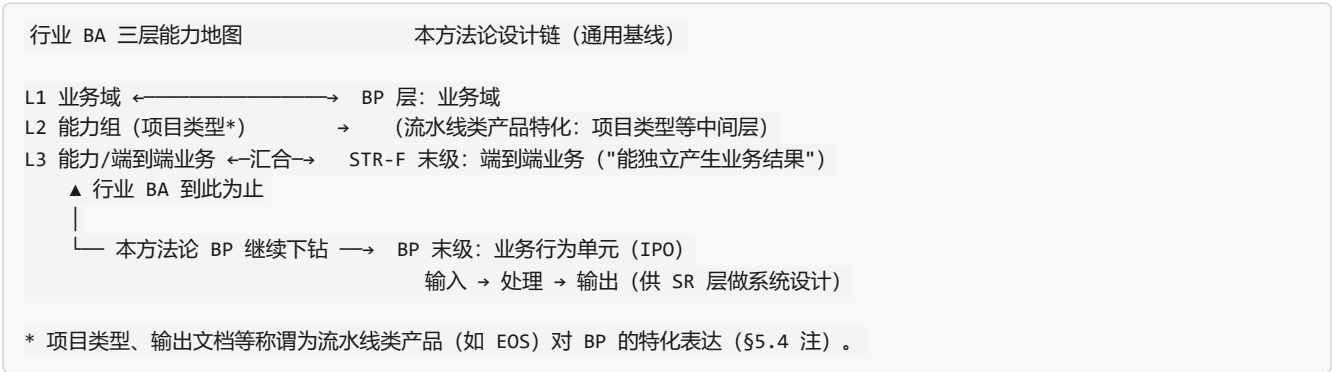
← L3 端到端业务：可独立产生"查勘任务分配结果"

← L3 端到端业务：可独立产生"查勘报告"

← L3 端到端业务：可独立产生"定损金额"

8.2.3 行业颗粒度与本方法论设计链的关系

将行业 BA 的三层能力地图，与本方法论设计链（通用规范 §4.4）放在一起对照，可以看到行业实践与本方法论的汇合点与分叉点：



行业实践和本方法论在**端到端业务**处汇合——"能独立产生业务结果"这个判定标准，映射到本方法论中就是 STR-F 末级节点：一个完整的业务闭环，有明确的输入和输出。

但本方法论没有停在这里。行业 BA 到此为止，本方法论的 **BP 继续下钻一级**——抵达业务行为单元（IPO）。为什么？

因为本方法论的 BP 不止是给管理层看的"能力全景图"——它还要给 SR 层（系统需求）做输入。SR 层需要知道的不只是"企业能做定损计算"，而是"定损计算这个端到端业务由哪些业务行为单元（IPO）构成，每个单元的输入是什么、输出是什么"。SR 层要支撑的不是一个笼统的"定损能力"，而是一个个具体的业务行为单元（输入→处理→输出）所要求的系统能力。

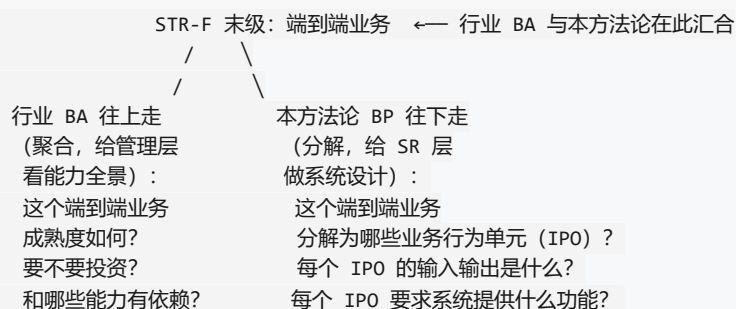
以"订单与合同"（STR-F 端到端业务）为例，本方法论 BP 将其展开为业务行为单元：



每一个 BP IPO 在 SR 层都会衍生出一组系统功能。如果 BP 停在端到端业务，SR 就无法确定到底要为哪些业务行为单元设计哪些功能——"订单与合同"太粗，SR 设计者不知道该设计几个功能模块、几个接口。

这就是本方法论 BP 比行业 BA 多下钻一级的原因：**行业 BA 的消费者是管理层（做投资决策），本方法论 BP 的消费者是 SR 层（做系统设计）。消费者不同，颗粒度自然不同。**

两者不矛盾——可以这样理解它们的关系：



同一个端到端业务锚点，两个方向。 行业 BA 从端到端业务往上聚合——管理层用它回答“我们企业的业务能力强不强、短板在哪、钱应该投在哪”。本方法论 BP 从端到端业务往下分解——SR 层用它回答“这个端到端业务由哪些业务行为单元构成、每个单元需要系统提供什么功能来支撑”。往上走的是能力热力图，往下走的是业务行为单元树。同一棵树的同一个分叉点，一个向上看树冠，一个向下看根系。

流水线类产品（如 EOS）的进一步特化： 本方法论的 BP 通用基线为「业务域 → IPO」两级。流水线类产品可在业务域与端到端业务之间增加「项目类型」「WBS」等中间层级，并将软件类产品的 IPO 输出形态表述为「输出文档」——该特化以产品级规范为准（§5.4 注、EOS 产品规范）。

8.3 业务能力的描述

8.3.1 核心字段

描述一个 L3 叶子能力，行业实践中通常用一组固定字段——比“名称+一句话”更结构化，但也不至于膨胀到一页纸：

字段	是什么	例子（定损计算）
能力名称	动词+名词，业务语言	定损计算
ID	唯一标识，层级编码	CLM-03-04
所属域	在能力地图中的父节点	理赔管理 > 查勘定损
定义	一句话说清这个能力做什么	基于查勘采集信息，按维修标准和配件价格数据库，计算出事故车辆的维修费用
业务结果	这个能力产出什么	定损金额及定损明细（工时+配件+辅料）
输入	触发和前置依赖	查勘报告、车辆损伤照片、维修价格库
关键业务规则	业务约束，不是技术规则	定损金额=工时费+配件费+辅料费；配件价格以厂家指导价为上限；超过车价70%推定全损
能力所有者	谁对这个能力的水平负责	理赔运营部
成熟度	当前水平	部分自动化——定损计算已规则化，但配件价格仍需人工核对

这不是数据库设计——不关心数据类型、不关心接口、不关心系统实现。只回答：这是什么能力、它做什么、产出什么、谁负责、水平怎么样。其中 **ID 和所属域是位置派生字段**——由能力在能力树中的位

置决定，不手工填写；真正需要填写的内容字段共七个：能力名称、定义、业务结果、输入、关键业务规则、能力所有者、成熟度。

8.3.2 不同场景用不同颗粒度

不是所有场景都需要全部字段。建能力地图时，第一轮只填名称+定义+所有者三个字段立起骨架即可；**成熟度、业务规则等字段在对应使用场景到来时再补**——比如管理层要看能力总览，成熟度就得先填上：

场景	需要的字段	不需要的字段
能力地图总览（管理层）	名称、定义、所有者、成熟度	输入、业务规则
能力差距分析（预算决策）	名称、成熟度、目标成熟度、差距	业务规则细节
系统覆盖分析（IT 规划）	名称、定义、支撑系统、覆盖程度	业务规则
变更影响分析	名称、依赖关系（输入来自哪些能力、输出流向哪些能力）	成熟度
新建能力（立项）	全量七字段 + 业务目标 + 成功标准	—

表格中的目标成熟度、差距、支撑系统、覆盖程度、业务目标、成功标准等为具体分析场景的**衍生字段**，不属能力定义的核心字段；「全量七字段」即 §8.3.1 扣除 ID、所属域两个位置派生字段后的七个内容字段。

一次要求全部填满是 BA 推不下去的首要原因。谁用、用来做什么、就补什么字段。

8.3.3 能力描述容易犯的三个错误

错误一：把"怎么做"写进了能力定义。

✖ "定损计算：查勘员在移动端上传照片，系统自动识别受损部件，调用配件价格库匹配价格……"

这是在描述流程和技术实现。能力的定义不应该绑定任何特定的"怎么做"——明天不用移动端了、用无人机拍照了，定损计算这个能力本身还在。

✔ "定损计算：基于损伤信息和价格标准，计算维修费用的能力。"

能力定义要稳定。流程会变、技术会变、组织会变——能力本身（企业需要能做这件事）不变。

错误二：把系统功能当成了业务能力。

✖ "OCR识别查勘单""自动匹配配件编码""GPS定位查勘员位置"

这些都是系统功能——主语是系统。业务能力的主语是企业。判定标准：**如果明天换了一套系统，这个东西还在吗？**定损计算——还在。OCR识别——可能换成了规则引擎或人工录入。前者是 BA 该描述的，后者不是。

错误三：层级混乱（见 §8.2.2）。

8.4 业务架构的五层价值

BA 的价值从近到远有五层，每一层解决一类不同的问题。

第一层：统一语义

同一个词，在不同部门含义不同。"客户"对市场部是"潜在购买者"，对风控部是"信用评分主体"，对财务部是"应收账款对象"，对 IT 部是 `customer` 表里的一条记录。四拨人天天开会，各自以为在说同一件事。

BA 做的第一件事不是画图，是**定名字**——把所有业务概念和能力统一命名并给出唯一定义，形成企业的业务语义公约。做完了这件事，跨部门沟通的摩擦成本就会明显下降。

第二层：能力差距分析

没有 BA 时，预算讨论长这样：

"明年的 IT 预算，销售部说要升级 CRM，运营部说要建数据中台，财务部说要换 ERP——各自报价，抓总的人判断不了轻重。"

有 BA 时，预算讨论变成：

"当前能力地图上有 247 项能力，其中 38 项不足、12 项缺失。这三个项目分别覆盖了哪些能力缺口？有没有三个项目都在补同一个缺口的情况？有没有哪个核心缺口完全没人提？"

BA 把散落的需求提案对齐到一张公共地图上，预算决策就有了结构——不是 BA 替你决策，是 BA 提供了决策的坐标系。

第三层：变更影响分析

一个业务规则变更——比如"客户分级从四级改成五级"——受影响的是客户信息管理（模型变了）、产品定价（新级别对应新价格策略）、风控规则（新级别风险敞口不同）、监管报告（分级口径变了）、前端界面（展示要调）。

没有 BA 时，这个分析靠开会——叫上各部门负责人，各自回去翻文档、问人，漏一个是一个。有 BA 时，从"客户分级"这个能力节点出发，沿依赖关系追踪，十几分钟出清单。不是 BA 替你判定了影响，是 BA 提供了不遗漏的检查起点。

第四层：去重

大型企业里，同一个业务能力被多个部门各自建设极其常见：客户身份验证——柜面有一套、网银有一套、信用卡中心还有一套；地址标准化——物流在做、风控在做、营销也在做。

BA 把能力地图拉出来，显性化这种重复——不强制合并，但至少让你看见"这个能力有三拨人在做"，然后做出知情决策。

第五层：战略到执行的翻译

管理层说"提升客户体验"——这对 IT 没有任何可操作性。BA 的翻译：客户体验涉及 14 项业务能力，其中 6 项成熟度低、3 项存在数据断点、2 项完全缺失。每项能力对应哪些系统、哪些团队、改造工作量多大。BA 把管理层的模糊意图翻译成能力级别的选项清单。

五层价值总结：

层次	价值	解决什么问题
语义	统一命名	"我们在说的是同一件事吗？"
差距	结构化评估	"钱应该投在哪里？"
变更	影响分析	"改了这里会波及什么？"
去重	显性化冗余	"有多少拨人在做同一件事？"
翻译	战略→执行	"管理层的这句话到底意味着什么？"

8.5 业务架构与相关方

业务相关方不关心架构方法论。他们关心的是：**这个东西能不能帮我解决我的问题？搞这个会不会给我添麻烦？** 让 BA 被业务方认可，以下几条是实践中反复验证有效的做法。

8.5.1 别让他们看到"架构"两个字

业务方不需要知道这叫"业务架构"。和他们讨论时，不要用"业务能力""架构视图""BA 建模"这些词。用他们的话："咱们先捋一下，你们业务上到底要管哪些事？每件事的产出是什么？哪些地方现在最难受？"

把他们的回答整理成能力图/文档后，再拿回去确认："上次聊的，我整理了一下，你看这是不是你们业务的全貌？有没有漏的？"——他们点头了，BA 就准了。你用什么方法论做的，不重要。

8.5.2 从痛点入手，不要从"全貌"入手

最让业务方反感的是："我们要做一个完整的企业业务架构，需要你们配合梳理全部业务能力。"——他们只觉得又多了一份填表任务。

从一个小切面入手。"你们最近最头疼的是什么？某个流程卡在哪？某个数据老出错？"顺着痛点倒推，整理出这块业务的能力结构，让他们看到——"哦，原来我们这块业务长这样，难怪那个地方老出问题。"**先解决一个具体问题，他们信了，再扩大范围。**

8.5.3 把 BA 变成他们的工具，不是 IT 的工具

BA 最大的失败模式是：IT 部门做了一套漂亮的业务架构，业务方看了一眼说"哦，挺好的"——然后继续用他们自己的 Excel 管理业务。

让业务方把 BA 用起来的关键：**BA 要能回答业务方自己的问题。** 业务负责人想做组织调整："我如果把 A 部门和 B 部门合并，哪些业务能力会受影响？"业务规划做明年计划："扩展线上渠道，现有能力哪些能复用、哪些要新建？"合规部门准备检查："监管要求的 X、Y、Z，我们当前覆盖情况怎么样？"如果 BA 能快速回答这些问题，业务方自然会用。

8.5.4 用他们的语言，用他们的节奏

- **不要让他们学 UML/BPMN/ArchiMate。** 你能用，他们不需要会。他们看懂的是表格、列表、简单的树形图。
- **不要追求完美。** 一张 80% 准确的能力地图，现在就能用。一张 100% 准确但三个月后才能出的，没用——三个月后业务已经变了。
- **更新要快。** 业务方提了一个变更，你当天或隔天就能同步到 BA 视图上。如果每次更新都要走评审流程、等两周，BA 就会变成又一个"档案"。

8.5.5 让他们有"拥有感"

最成功的 BA 不是 IT 部门"交付"给业务方的，而是业务方觉得自己"参与构建"的。实操差异：

✖

"我们是架构团队，需要你们配合做业务能力梳理，请安排人参加为期三天的 workshop。"

✔

"你们上次说业务扩张后流程越来越乱，我整理了一套方法可以帮你们把业务结构理清楚。我们先花一小时试试，如果没用就算了。"

第一种是**他们帮你做事**，第二种是**你帮他们解决问题**。同一件事，完全不同的感受。

8.6 用业务架构提升能力成熟度

BA 提升能力成熟度，不是一次性的"做完架构然后去提升"，而是一个持续循环：**评估 → 定位 → 投入 → 度量 → 重评估**。BA 的角色是让这个循环有结构、有优先级、可度量。

8.6.1 能力成熟度模型

行业实践中常用五级：

级别	叫什么	含义	标志性问题
1	初始/临时	靠人撑着，没有标准做法，换人就断档	"这事只有老王会"
2	标准化	有明确流程、有岗位定义、可重复执行	"换个人培训两周能上手"
3	流程化	流程被定义、被遵守、被度量	"我能说出这个能力的效率数字"
4	自动化	大部分操作由系统完成，人做异常处理	"系统自动完成，人只看异常"
5	优化驱动	能力本身在持续自我改进	"能力水平在主动提升"

不是所有能力都需要到 5 级。 一个支持性能力（如"档案管理"）到 3 级就够；一个差异化能力（如保险公司的"定损计算"）可能需要到 4 级、甚至 5 级。BA 的价值不是推所有能力往高级别冲，而是识别**哪些能力值得高成熟度**。

8.6.2 循环一：评估——全量成熟度扫描

第一轮，对所有 L3 叶子能力做快速自评。不需要精确，三级精度足够，做完全量大概几个工作日。产出物是一张**能力成熟度热力图**：

	成熟度低 ← 定损计算	成熟度中 核赔	成熟度高 —
差异化能力 (决定竞争力)			
核心运营能力 (不出事的前提)	保单校验	—	报案受理
支持性能力 (必要但不区分)	追偿	—	档案管理

这张图本身就是沟通工具——最决定竞争力的能力恰好成熟度最低，而支持性能力倒是挺高。这不合理。但没这张图之前，这个不合理是隐形的。

8.6.3 循环二：定位——区分三种差距

评估完之后最常见的错误是：把所有低分能力列出来，从低到高排序，逐个推。这是把复杂度问题当成排序问题。实际上有三种完全不同性质的差距：

差距类型	特征	应对策略	节奏
A：关键能力薄弱	差异化/竞争力能力，成熟度 ≤ 2	集中资源，单点突破	重点项目，季度见效
B：核心能力有断点	各自 ≥ 3 级，但上下游交接断裂	打通接口和数据	端到端优化
C：大量低成熟度	初创/野蛮生长期常态	先建标准化框架，批量提升	制度化推进，年度见效

区分差距类型是 BA 提供的核心判断框架。类型 A 需要投资，类型 B 需要协同，类型 C 需要制度——应对逻辑完全不同。

8.6.4 循环三：投入——预算请求映射到能力节点

业务部门提预算请求时，BA 强制一个动作：**每个预算请求必须映射到具体的能力节点。**

以前：

销售部："我们要升级 CRM，预算 500 万。"——这句话没法判断值不值。

现在：

BA 对照能力地图："CRM 升级主要影响三项能力——客户信息管理 (L2→L3)、销售线索管理 (L2→L3)、客户分级与画像 (L1→L3)。" 决策者："客户分级与画像是战略重点，但客户信息管理从 L2 到 L3 值得花这个钱吗？"

BA 把模糊的"升级系统"翻译成了具体的能力改造选项，让决策者能在同一语义框架下比较：这个投资提升了什么能力、从什么水平到目标什么水平、值得吗。

8.6.5 循环四：度量——用业务指标，不用架构指标

BA 项目最常问的指标是错的——"架构覆盖率" "能力模型完整度"。业务方不关心这些。应该度量的是能力本身的业务指标：

能力	当前指标	目标	度量方式
定损计算	定损周期 2.3 天，差错率 8%	周期 < 1 天，差错率 < 2%	系统记录，月度统计
报案受理	自动登记率 60%	自动登记率 > 90%	系统日志
追偿处理	成功率 12%，平均回款 14 个月	成功率 > 25%，回款 < 9 个月	财务数据

业务方不看"成熟度从 2 级提到了 3 级"，他们看"定损周期从 2.3 天降到了 1 天"。成熟度是给架构师用的管理工具，业务指标是给业务方看的成果。两者必须对应。

8.6.6 循环五：重评估——闭环与学习

一个周期（通常半年到一年）后，重新评估全量能力。两个关键动作：

检查投入产出。 年初投了三个能力改造项目，年底成熟度实际提升了多少？业务指标改善了没有？哪些投入见效、哪些打了水漂？这个复盘直接决定下一轮资源分配的优先级。

校准目标。 外部环境变了——去年"定损计算 L4"是合理目标，今年 AI 技术进步、行业标准提高了，目标可能需要重新设定。BA 的循环机制保证了目标不会锁定在去年的判断上。

最被低估的环节是重评估。 大部分 BA 项目在第一轮热力图做完后就停滞了。但真正让 BA 发挥持续价值的不是第一张图，而是第三年的第六张图——它告诉你哪些投入有效、哪些是浪费、能力的自然衰减速度是多少。这个学习积累才是企业最稀缺的架构知识。

8.6.7 三类提升的具体路径

不同性质的能力，提升路径完全不同：

阶段	典型问题	BA 的作用	常见阻力
标准化（1→2）	同一个能力在不同团队做法完全不同	定义统一的能力标准和业务规则，建立能力所有者的问责	"我一直这么做的，不也挺好"——标准化不是否定经验，是把经验提炼成可传递的知识
流程化（2→3）	有标准流程但不知道执行质量	为每个能力定义 2-3 个关键业务指标	"我们的工作没法量化"——总有办法量化，挑一个最敏感的指标先开始
自动化与优化（3→4→5）	效率瓶颈在人力	确保自动化改造不破坏能力的业务边界；跨能力共享不应耦合业务逻辑	—

附录 A：本文档与通用系统设计规范的关系

本文档（07-general-spec-architecture-fundamentals.md）与通用系统设计规范（01-general-spec-sysdev.md）是互补关系：

	架构基础概念与方法	通用系统设计规范
定位	概念入门和原理阐述	方法论操作规范和规则

	架构基础概念与方法	通用系统设计规范
内容	系统/架构/架构决策/架构视图的定义和原理	五层设计链的操作步骤、文档结构、验收标准
读者	任何需要建立架构认知的人	使用本方法论的从业者
依赖	不依赖任何方法论	其基本概念 (§1.1) 可引用本文档作为扩展阅读

本文档中涉及五层链的讨论（第五章）以通用系统设计规范中的定义为准，本文档仅为说明原理而引用，不代表对五层链定义的修改。

附录 B：术语速查

术语	英文	一句话定义
系统	System	一组相互关联的元素为达成共同目的而构成的整体（综合 ISO 9000 / ISO 15288 / 一般系统论）
架构	Architecture	系统的基本概念或属性，体现于元素、关系及原则（ISO 42010）
架构描述	Architecture Description	表达架构的文档/制品集合
架构视角	Architecture Viewpoint	一个观察角度——规定了关注点、相关方和模型类型
架构视图	Architecture View	从一个特定视角对架构的描述
架构决策	Architecture Decision	选错了后来要花很大代价才能改对的选择
架构风格/模式	Architecture Style/Pattern	架构决策的套路（如微服务、事件驱动、分层），不是架构本身
不可逆性	Irreversibility	修改成本高到理性决策者会选择忍受不完美的程度（经济属性）
相关方	Stakeholder	与系统有利益关系的个人、团队或组织
关注点	Concern	相关方对系统关心的某个方面
涌现属性	Emergent Property	来自元素间关系而非单个元素的系统属性（如可靠性、可扩展性）
康威定律	Conway's Law	系统结构是组织沟通结构的副本